AVL

Un AVL est un BST doté de propriétés supplémentaires, adaptées à l'efficacité des algorithmes.

Le nom AVL vient de Georgii **A**delson-**V**elsky et Evgueni **L**andis , qui ont publié un article sur cette structure de données en 1962.

AVL

Propriétés :

- Un AVL est un BST
- C'est un BST "équilibré".
- En raison de sa structure, la complexité des opérations est bonne.
- Opérations : insérer, supprimer, rechercher, équilibrer

AVL

Propriétés :

- Un AVL est un BST
- C'est un BST "équilibrée".
- En raison de sa structure, la complexité des opérations est bonne.
- Opérations : insérer, supprimer, rechercher, équilibrer

Les condition(s) :
`pn->valeur > pn->left>valeur`
et
`pn->valeur < pn->right->valeur`
sont encore vraies

AVL

Propriétés :

- Un AVL est un BST
- C'est un BST "équilibrée".
- En raison de sa structure, la complexité des opérations est bonne.
- Opérations : insérer, supprimer, rechercher, équilibrer

Maintenir l'arbre "équilibré" après chaque opération afin de **minimiser** sa hauteur globale.

AVL

Propriétés :

- Un AVL est un BST
- C'est un BST "équilibrée".
- En raison de sa structure, la complexité des opérations est bonne.
- Opérations : insérer, supprimer, rechercher, équilibrer

Comme la complexité des opérations sur un arbre dégénéré est celle d'une liste, elle est *O(N)*.

Nous voulons atteindre un meilleur niveau de complexité....un niveau logarithmique

AVL

Opérations de rotation

Nous ajoutons maintenant un autre type d'opération pour modifier la structure d'un arbre : les rotations.

Les rotations sont des opérations qui nécessitent :

- un nœud (en fait un pointeur `t_node *`) à partir duquel appliquer la rotation
- Une "direction" : soit "gauche", soit "droite".

AVL

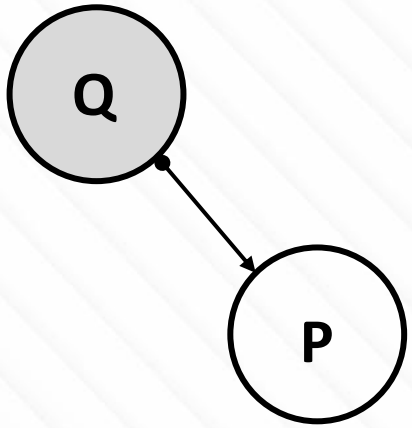
Opérations de rotation

Soit Q un nœud, nous voulons lui appliquer une rotation

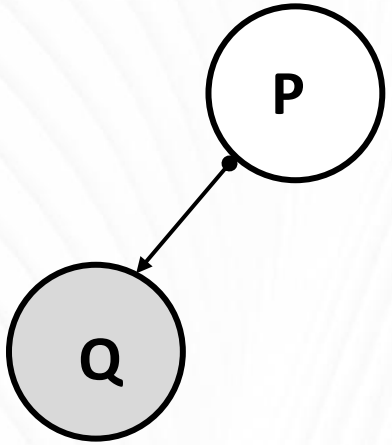
Premier effet : l'un des fils de Q deviendra son "parent".

pour la rotation à gauche :

avant

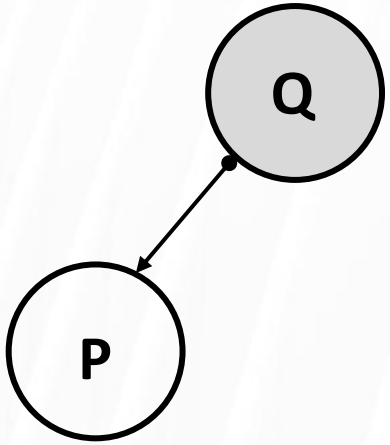


après

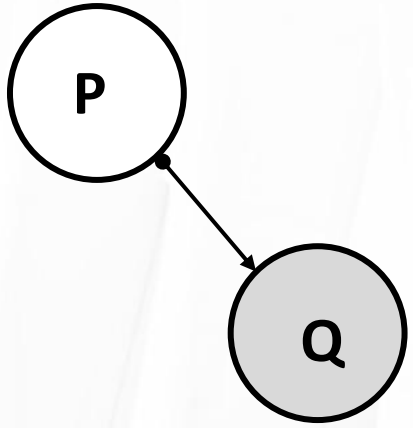


pour la rotation à droite :

avant



après

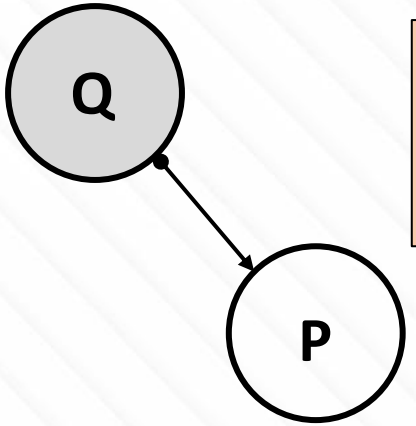


AVL

Vérifions : s'agit-il toujours d'un BST ?

pour la rotation à gauche :

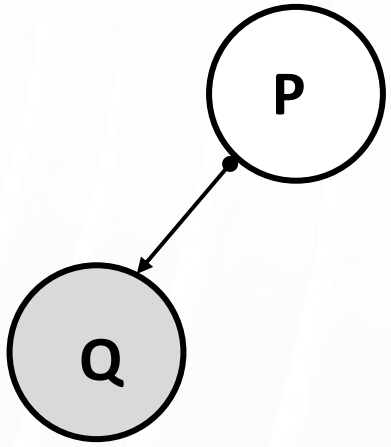
avant



P est le fils droit de Q

$P > Q$

après



Q est le fils gauche de P

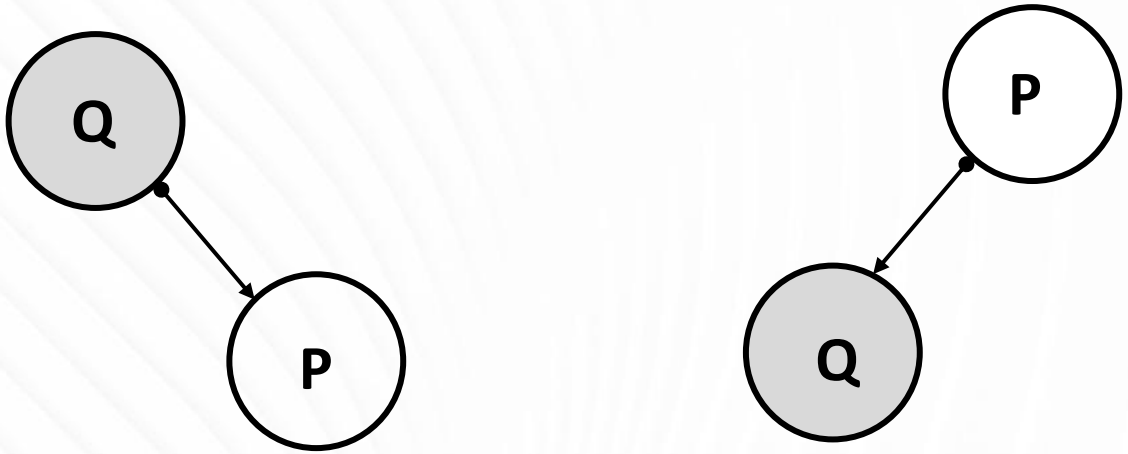
$Q < P$

Ainsi, la rotation à gauche conserve la propriété du BST - même chose pour la rotation à droite

AVL

Un aide-mémoire

pour la rotation à **gauche** : Q devient un fils **gauche**



Mais ici, nous changeons la structure de l'arbre....

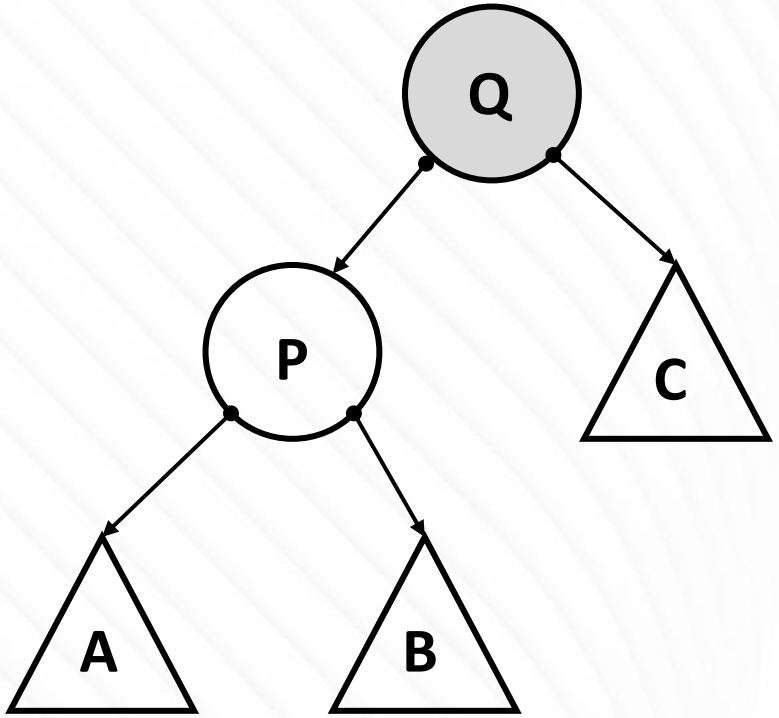
P était le fils droit de Q..... Qui est maintenant le fils droit de Q ?

Q est maintenant le fils gauche de P...qu'en est-il du fils gauche de P avant la rotation ? ???

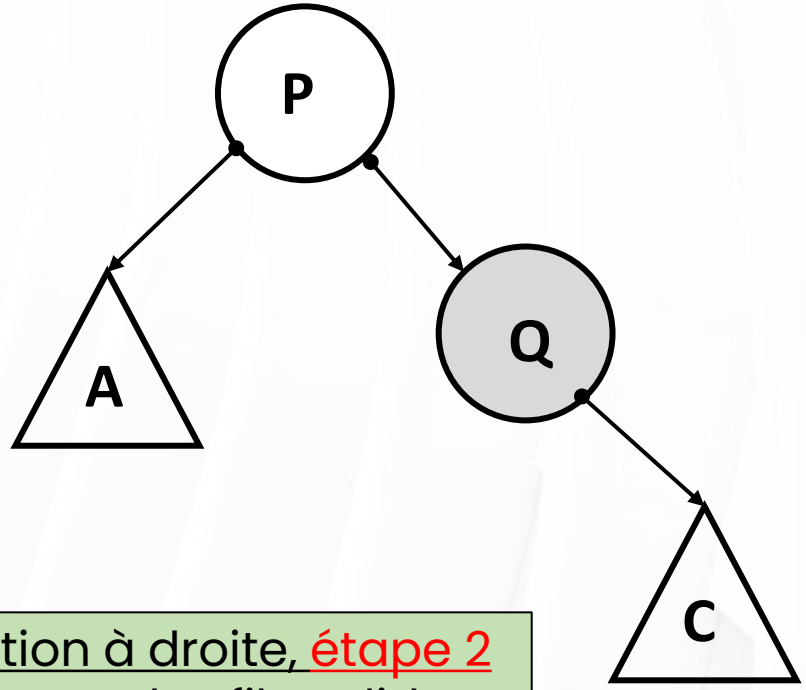
AVL

Illustration pour une rotation droite

Considérons la situation suivante : Q et P sont des nœuds, A, B et C sont des sous-arbres (ils ne changeront pas).



Rotation à droite, étape 1
Q devient un fils droit

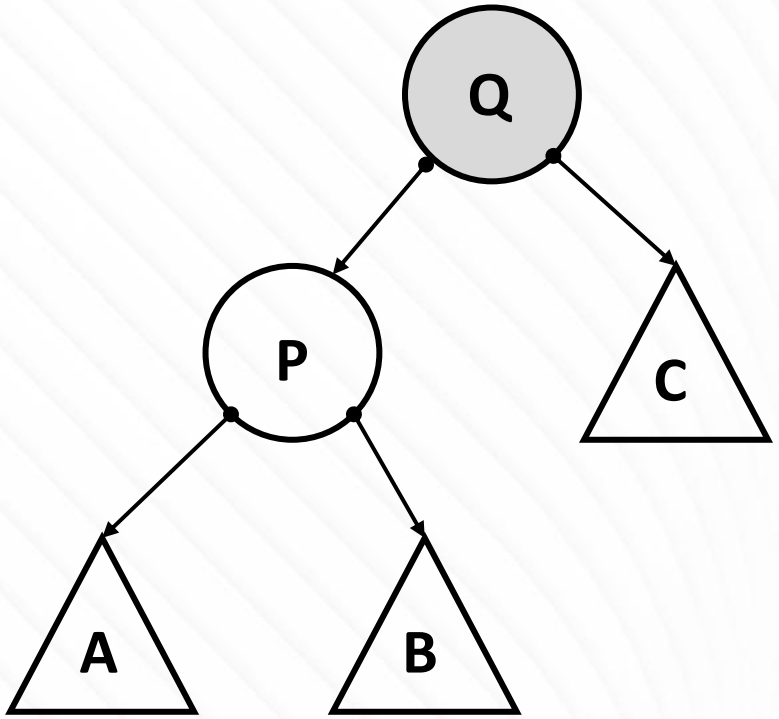


Rotation à droite, étape 2
Conserver les fils valides

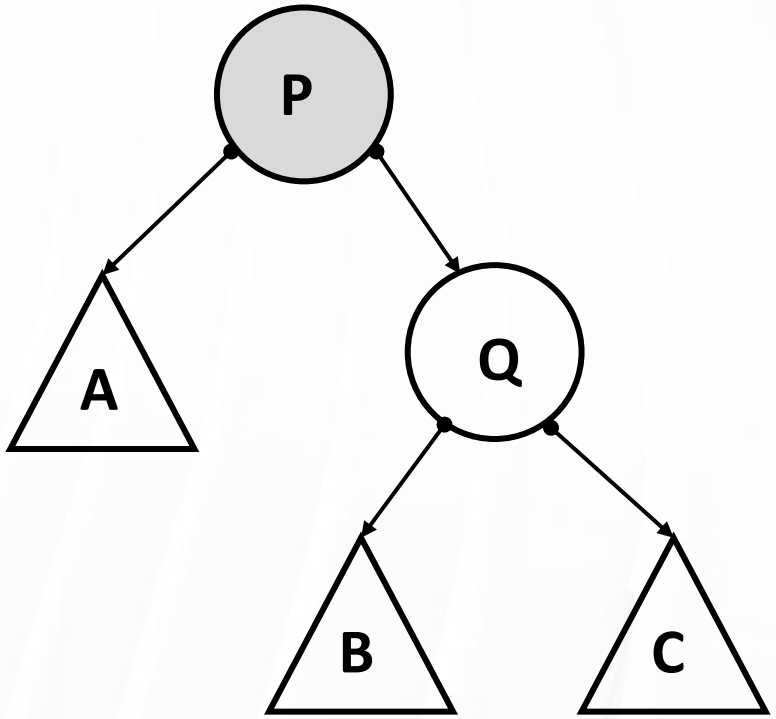
Rattacher B

A partir de la configuration d'origine :

Les valeurs de B sont "supérieures" à P
Les valeurs de B sont "inférieures" à Q



Les valeurs de B sont "inférieures" à Q
Les valeurs de B sont "supérieures" à P



Instructions correspondantes

Dans cet exemple, `root` pointe vers Q

```
void rightRotation(t_node *root)
{
      t_node *pivot = root->left ;
      root->left = pivot->right ;
      pivot->right = root ;
      root = pivot ;
}
```

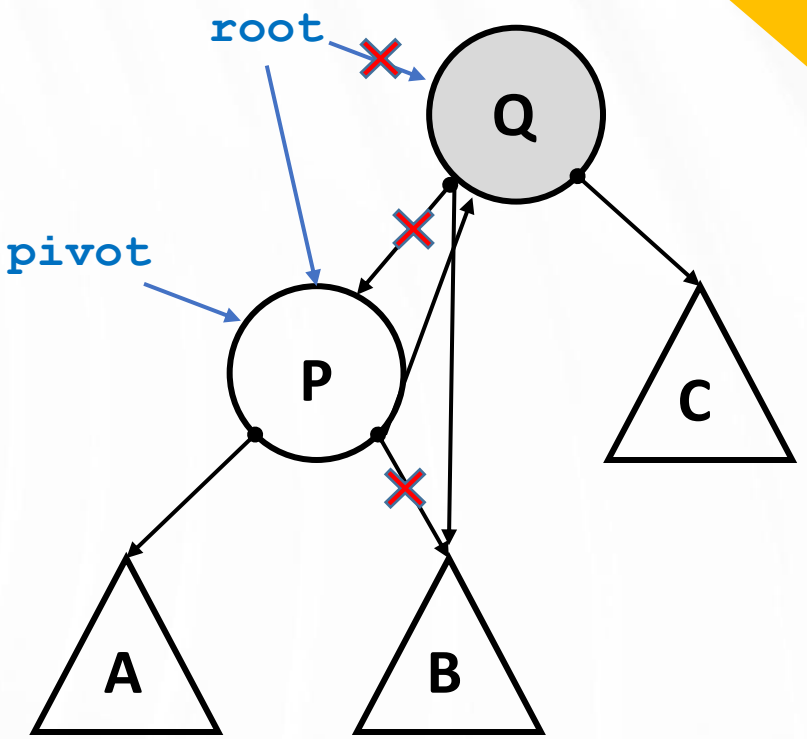
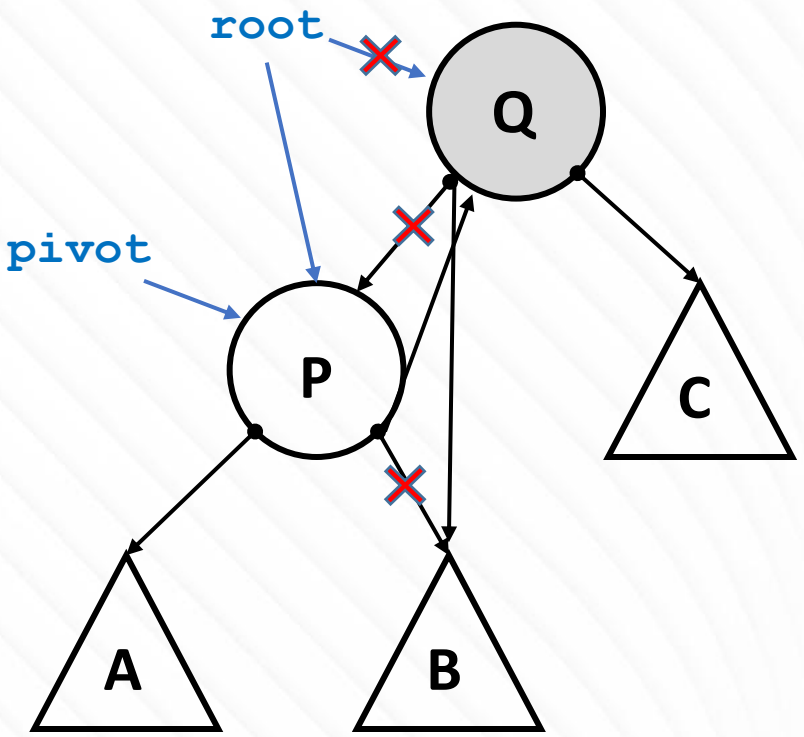
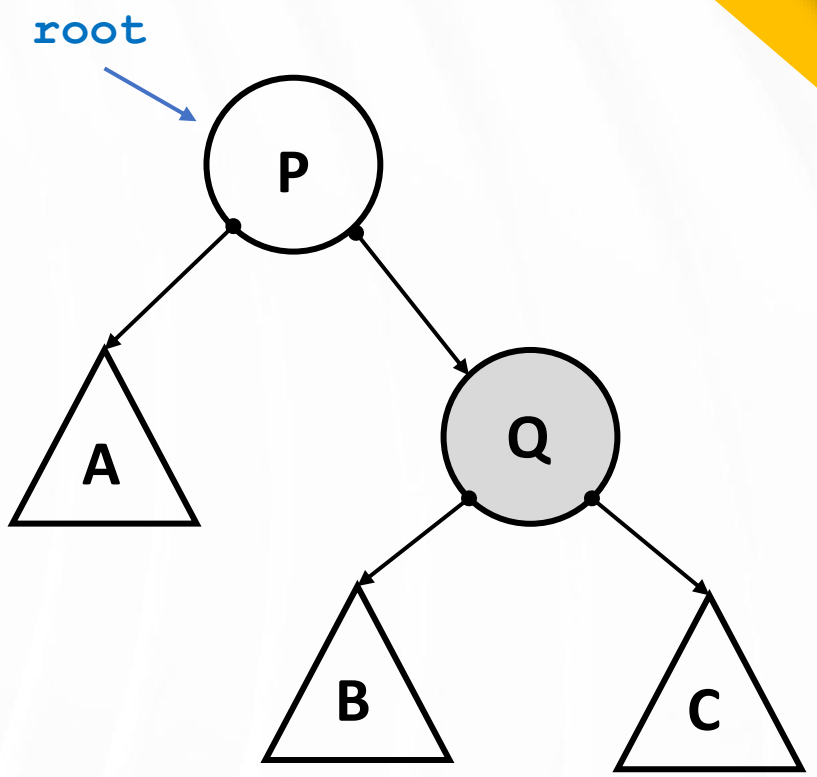


Illustration à partir de la nouvelle racine





Quand appliquer les rotations ?

A chaque insertion d'une valeur

Chaque insertion crée une **feuille** (vers le ‘bas’ de l’arbre’)

La profondeur de l'arbre / des sous-arbres peut changer

Cadre global pour maintenir une BST en tant qu'AVL :

insérer une valeur dans un AVL = insertion standard BST + équilibrer l'arbre

Facteur d'équilibre pour les nœuds

Le facteur d'équilibre est lié à la hauteur :

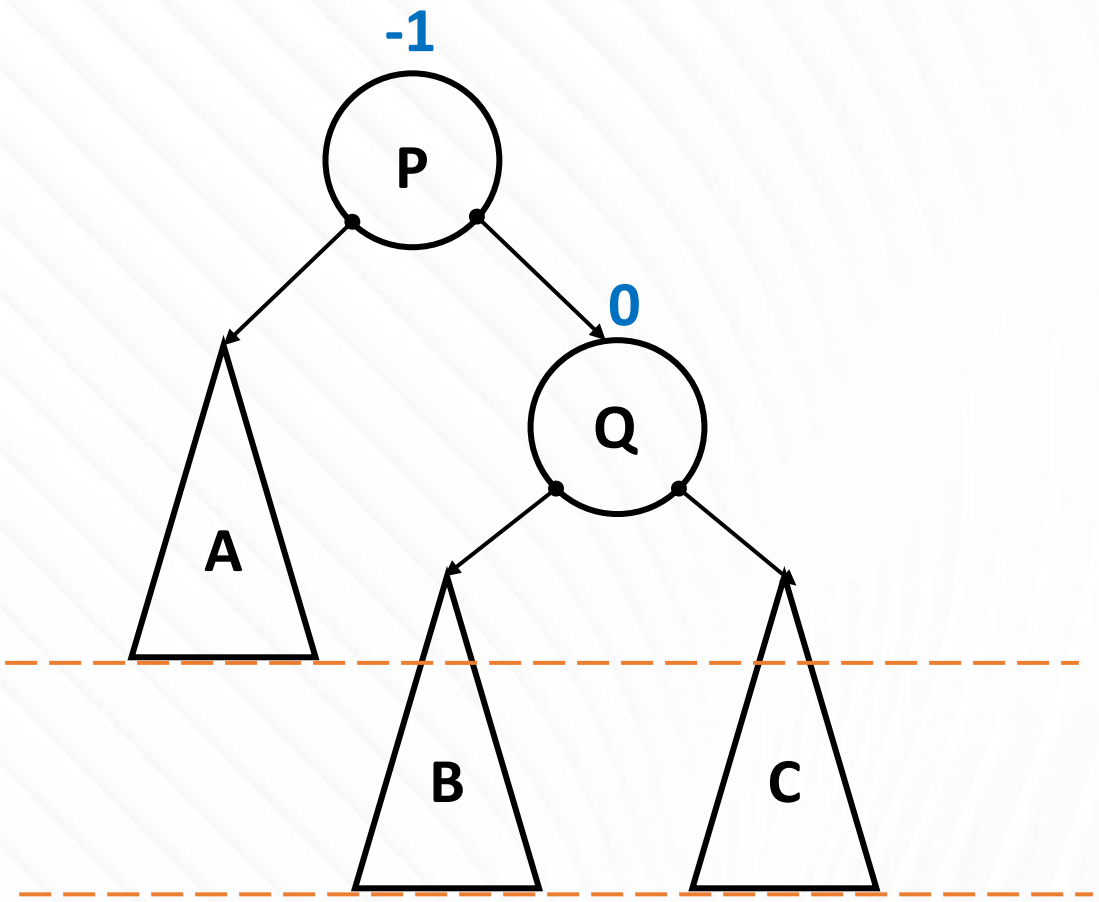
Le facteur d'équilibre (FA) d'un nœud est égal à :
la hauteur de son sous-arbre gauche – la hauteur de son sous-arbre droit

C'est une information **locale** à un nœud.

Un arbre est équilibré si **tous** ses nœuds ont un facteur d'équilibre de -1, 0 ou +1.

La hauteur d'un arbre (= de son nœud `root`), seule, ne dit rien de sa structure détaillée

Exemples

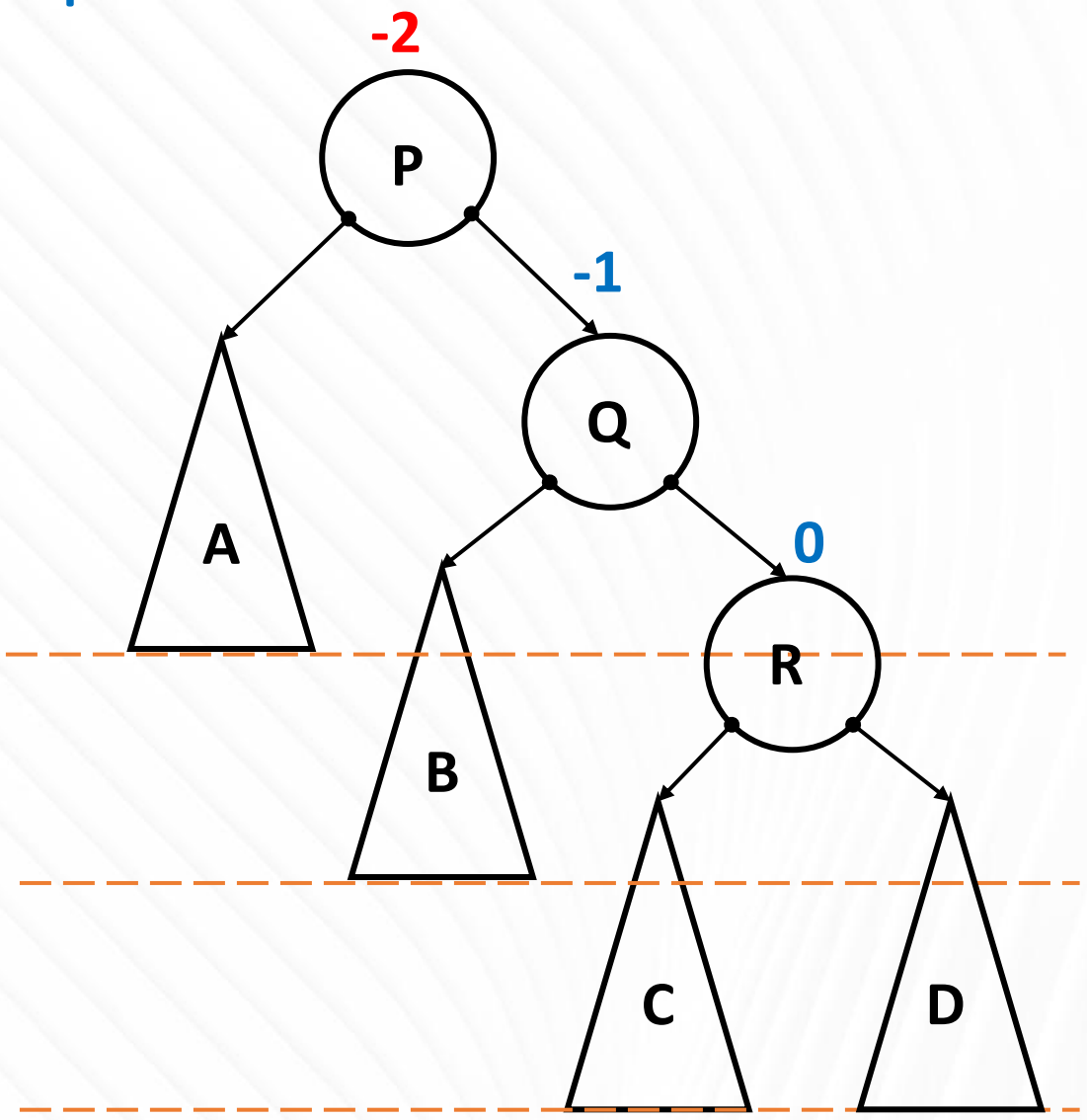


Facteurs d'équilibre pour P et Q :

L'arbre est équilibré

-1 pour P : le sous-arbre de droite est plus "profond" que celui de gauche

Exemples



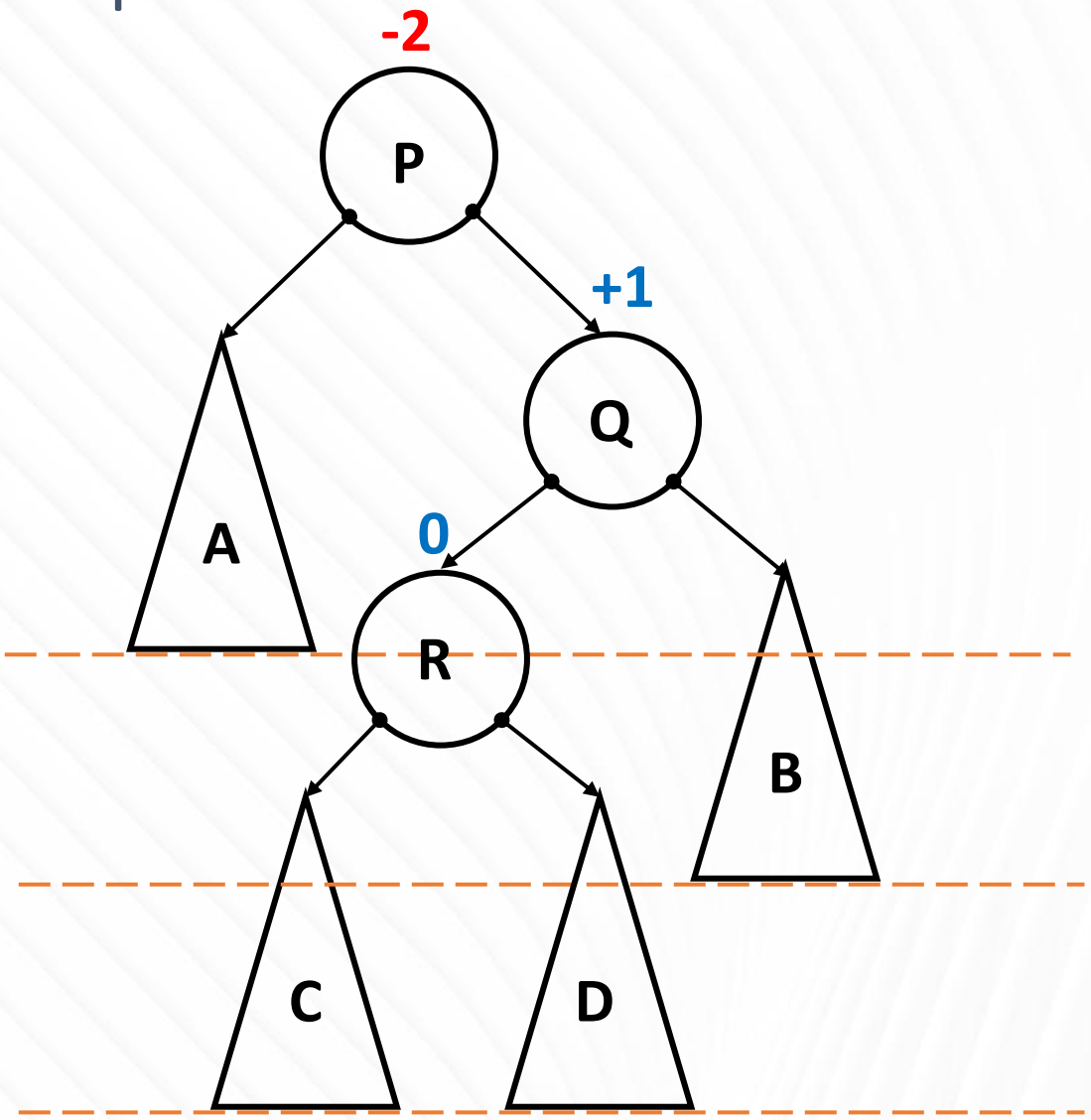
Facteurs d'équilibre pour P, Q et R :

L'arbre n'est pas équilibré

-2 pour P :Le sous-arbre droit de P est "trop" profond

-1 pour Q :Le sous-arbre droit de Q est plus profond que son sous-arbre gauche

Exemples



Facteurs d'équilibre pour P, Q et R :

L'arbre n'est pas équilibré

-2 pour P :Le sous-arbre droit de P est "trop" profond

+1 pour Q : Le sous-arbre gauche de Q est plus profond que son sous-arbre droit

Comment équilibrer un arbre ?

Si le facteur d'équilibre d'un nœud `pn` est de -2 :

Vérifier le facteur d'équilibre de `pn->right` :

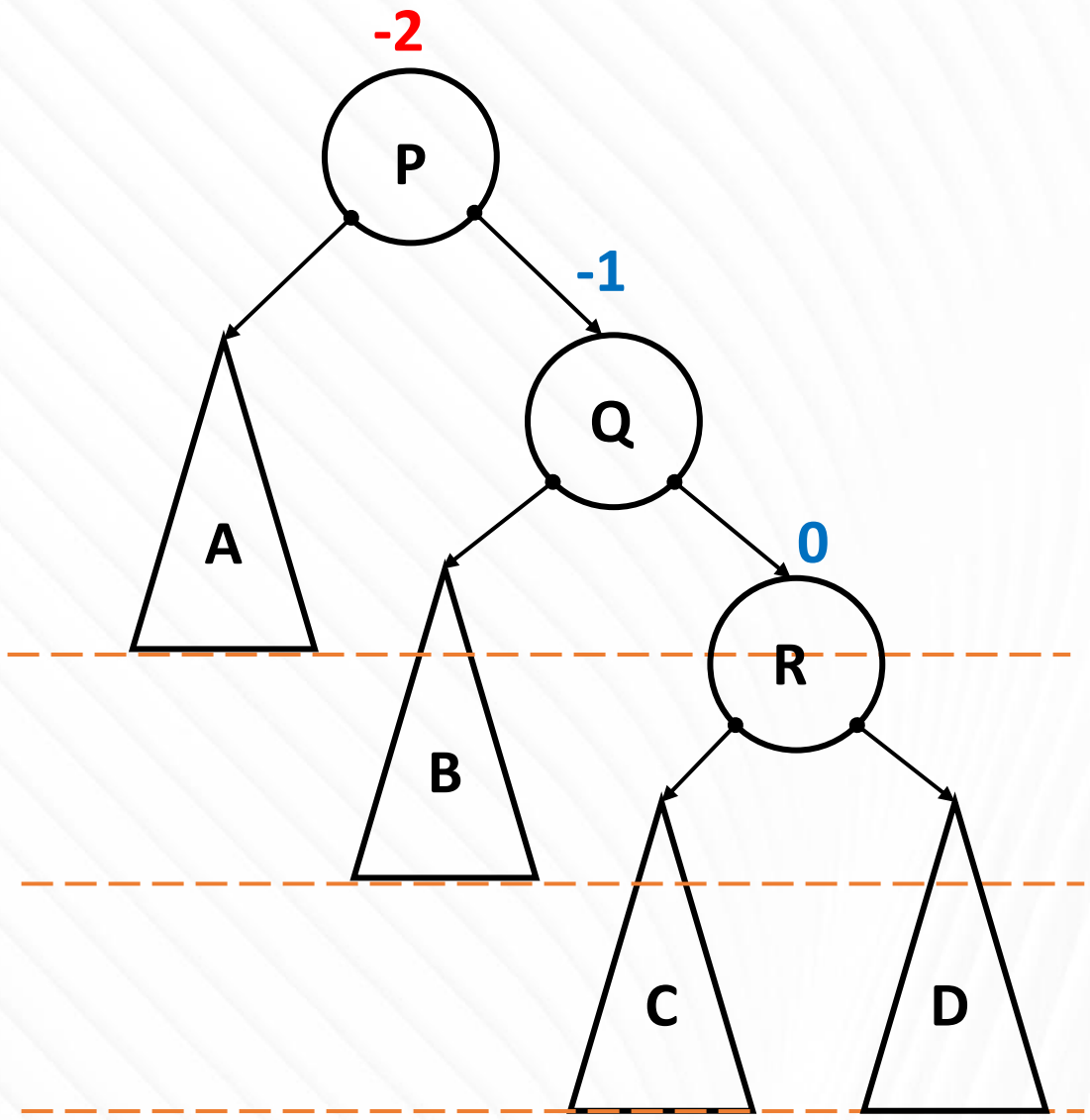
S'il est +1 : effectuer une opération d'équilibrage

S'il est égal à -1 : effectuer une autre opération d'équilibrage

Quelle opération d'équilibrage ?

La seule que nous connaissons : effectuer des rotations

Essayons avec le premier exemple

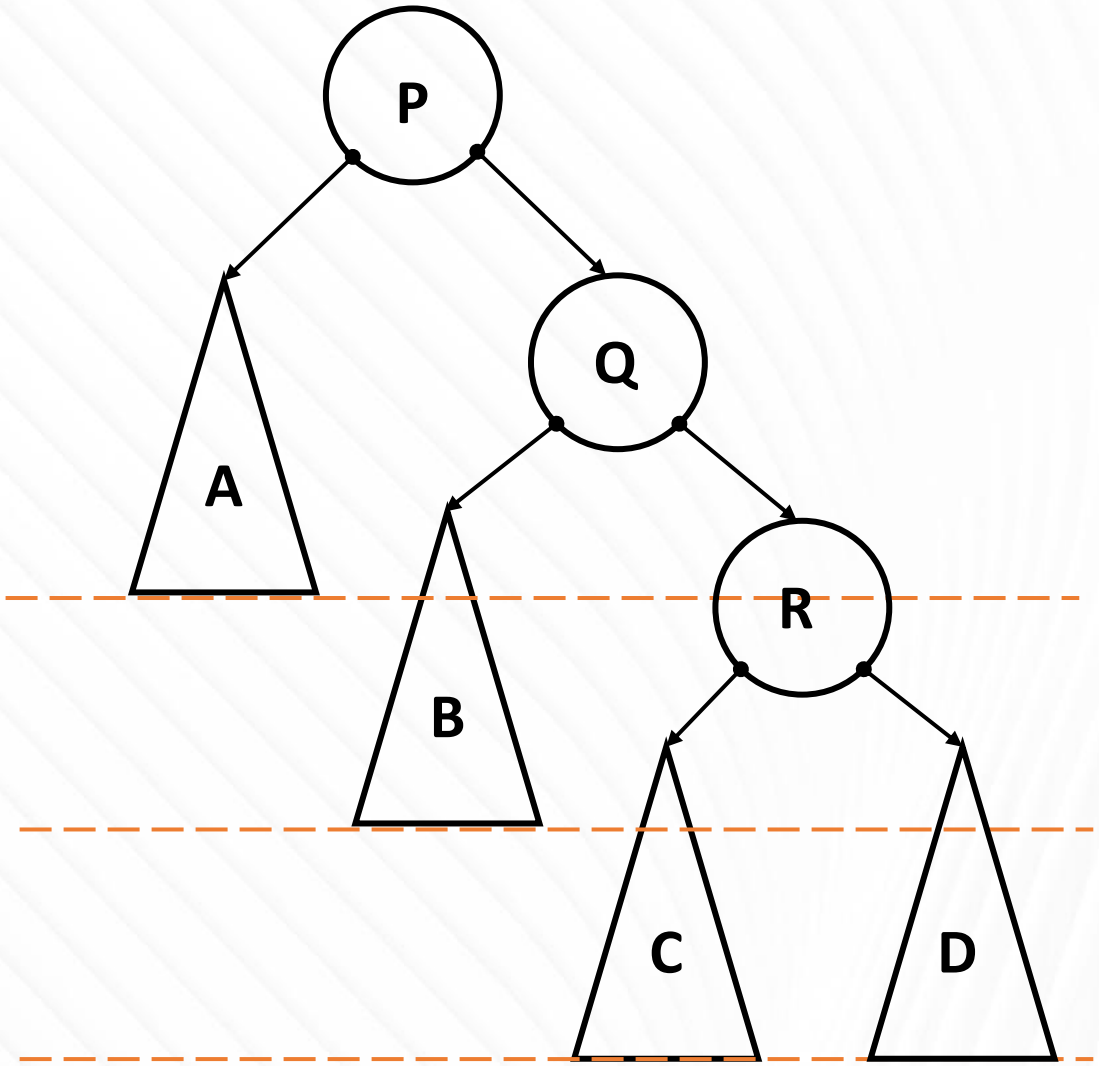


Déséquilibre droit de P
Le sous-arbre droit de Q est 'plus profond'

Fonctionnement ?

Rotation à gauche sur P

Illustration pas à pas de la rotation à gauche sur P



Étape 1 :

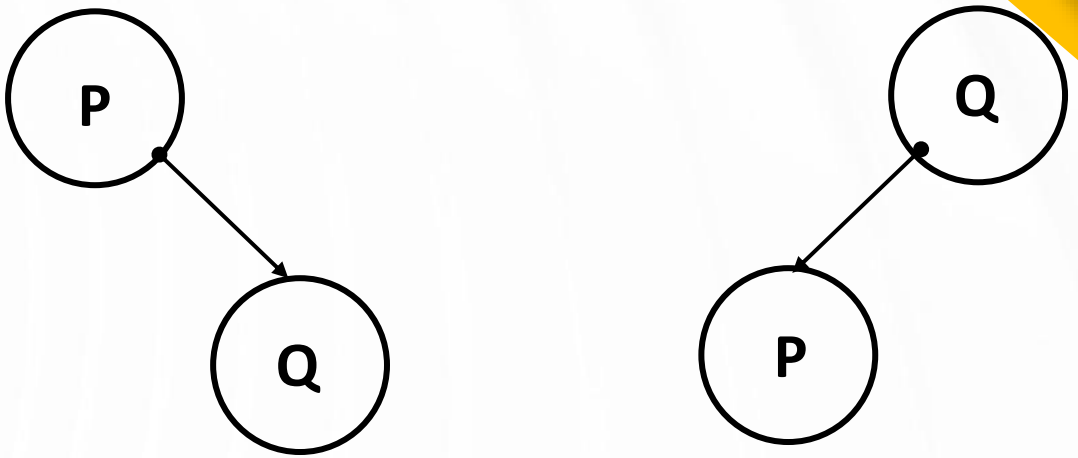
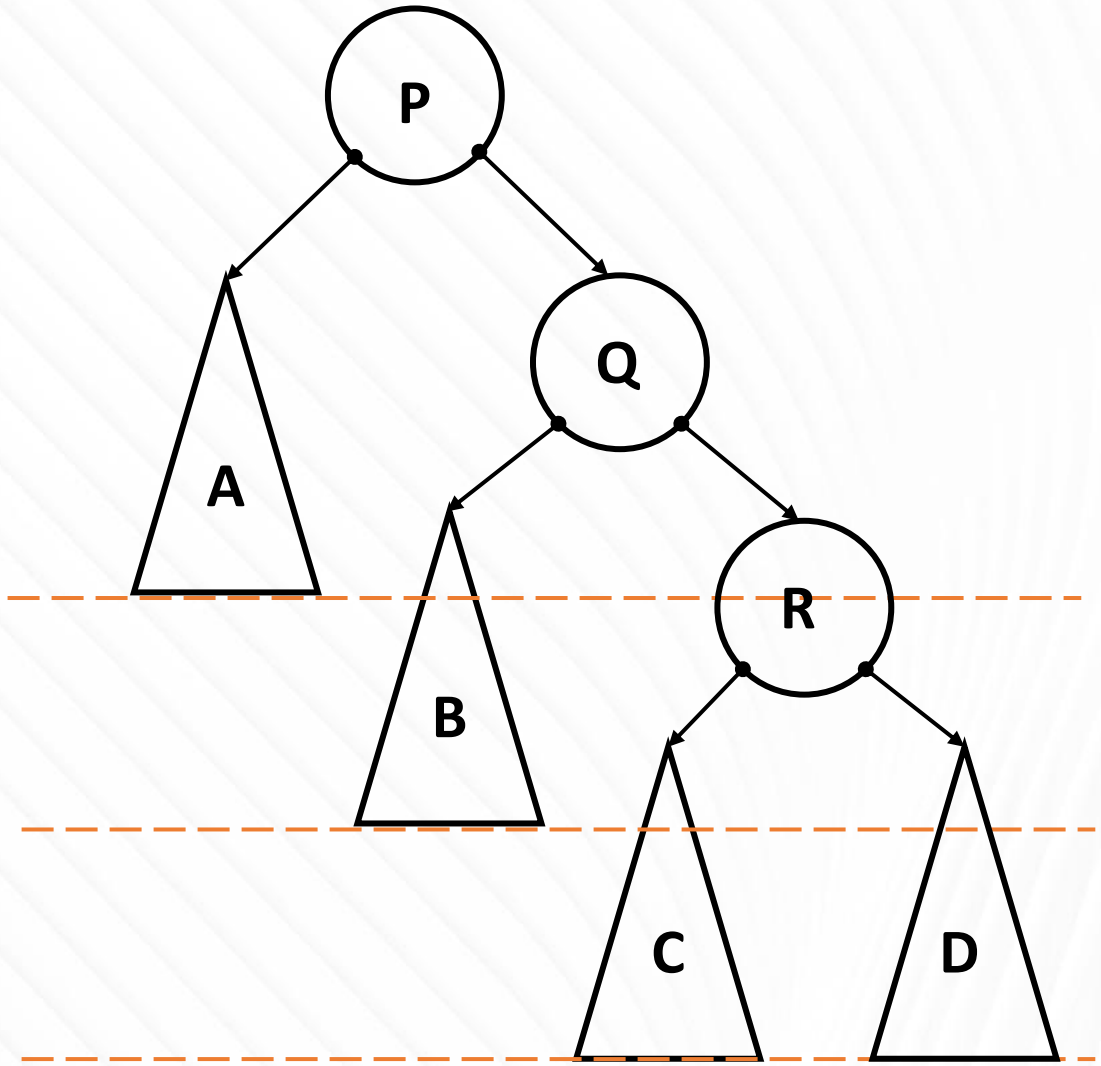


Illustration pas à pas de la rotation à gauche sur P



Étape 2 : conserver ce qui peut l'être à l'endroit où il se trouve

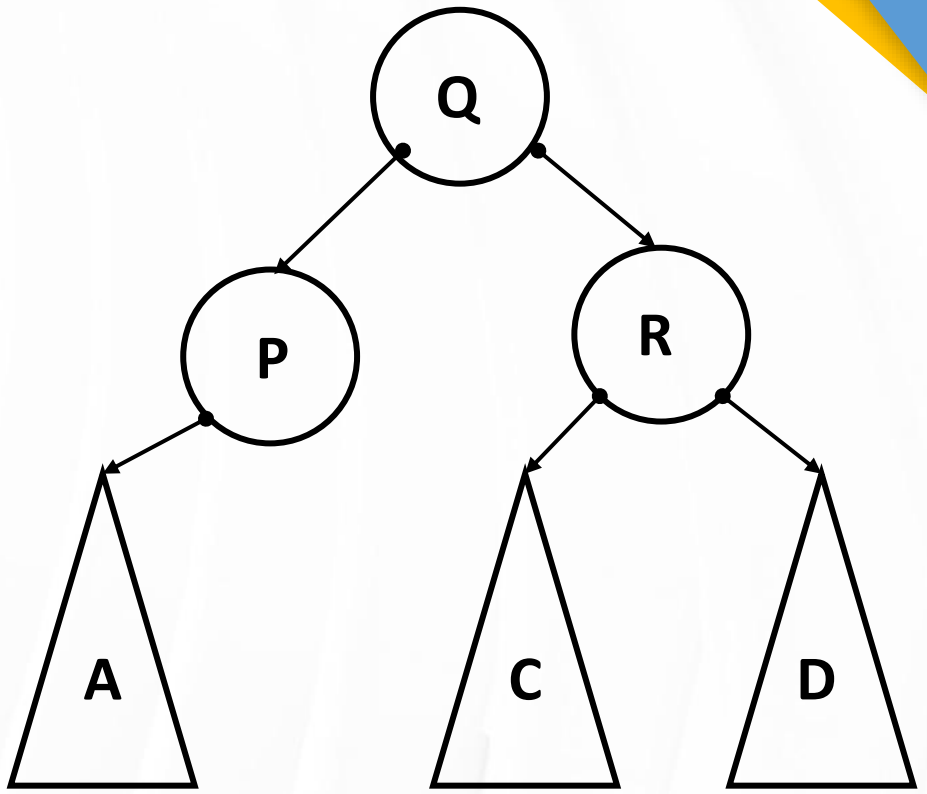
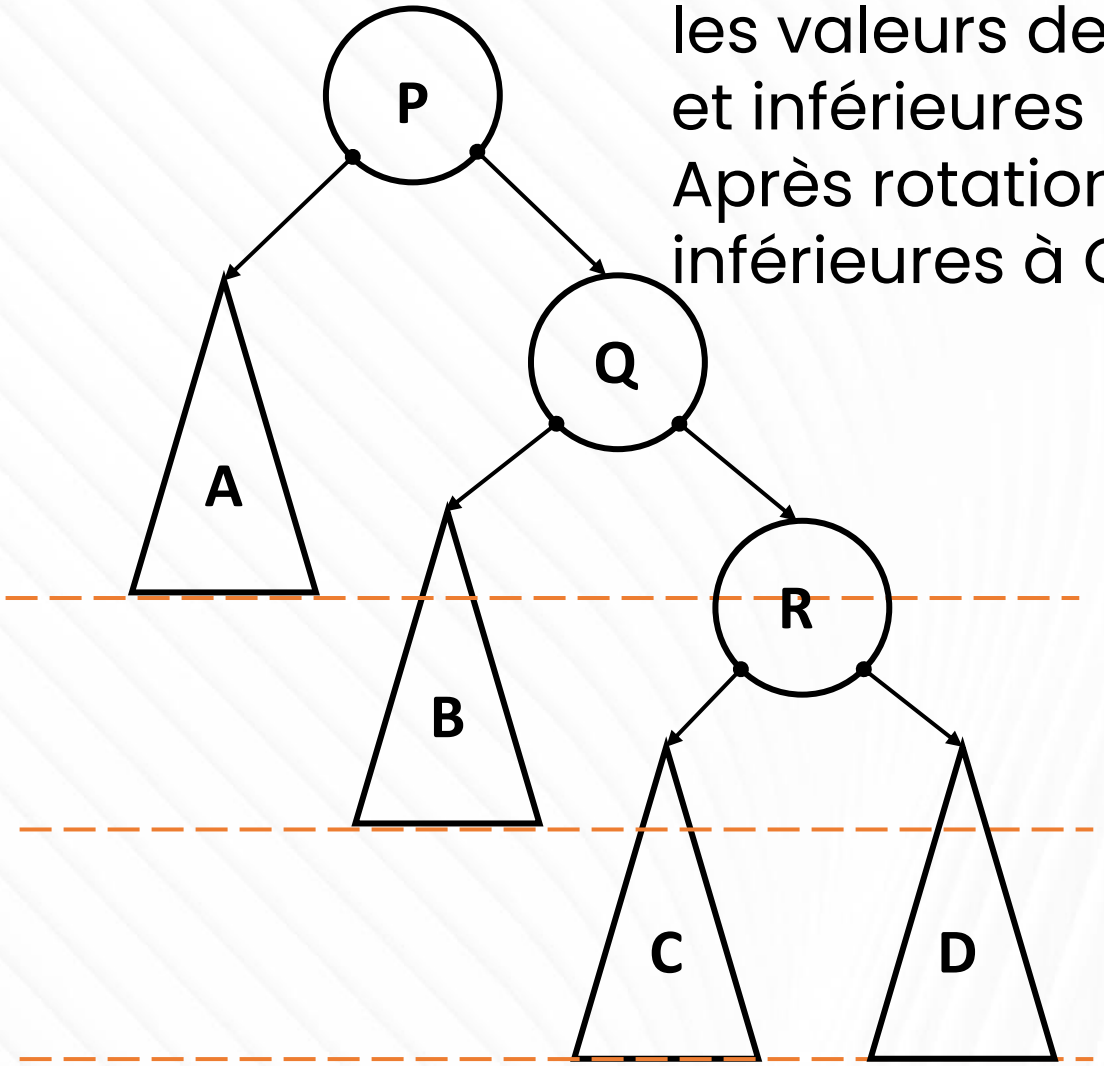
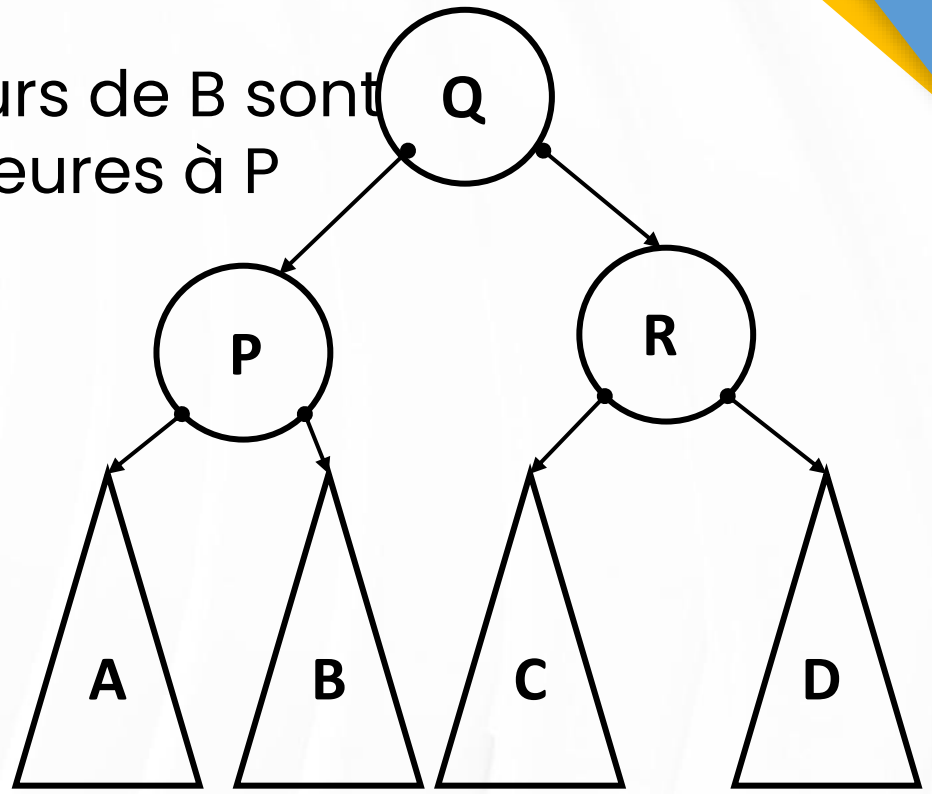


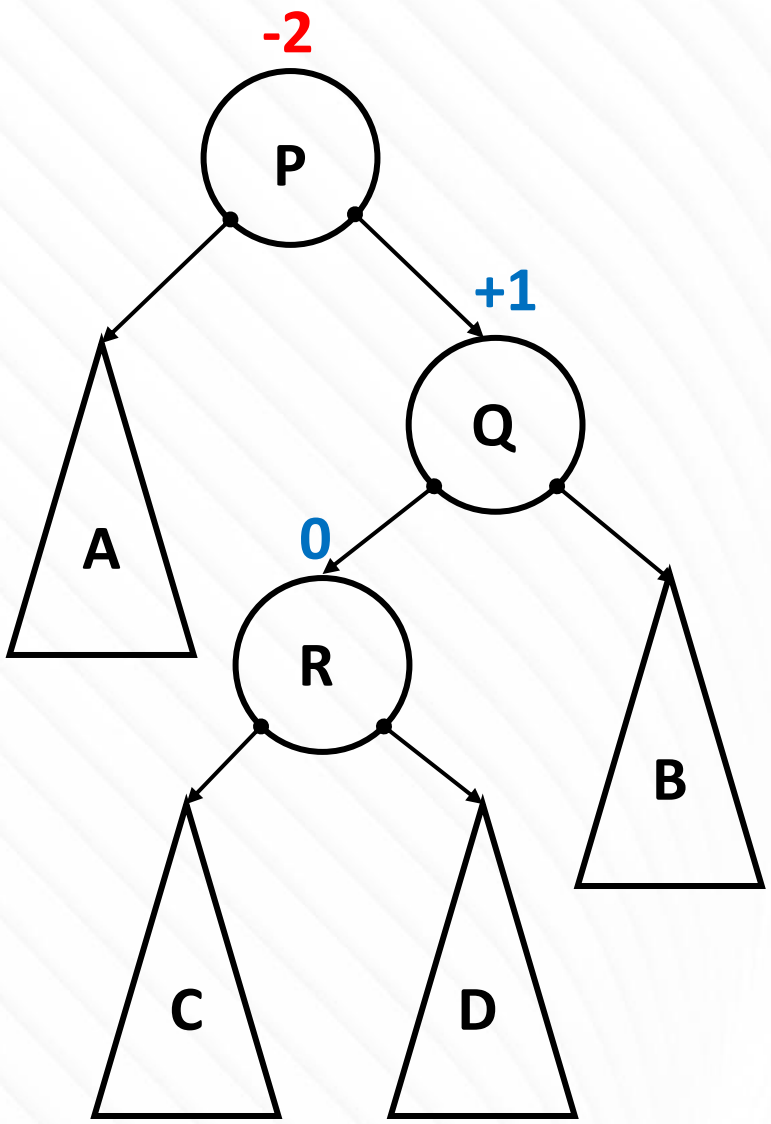
Illustration pas à pas de la rotation à gauche sur P

A partir du BST non équilibré :
les valeurs de B sont supérieures à P
et inférieures à Q
Après rotation : les valeurs de B sont
inférieures à Q et supérieures à P





Essayons avec le deuxième exemple



Déséquilibre droit de P
Le sous-arbre gauche de Q est 'plus profond'

Fonctionnement ?

Double rotation à gauche sur P

Rotation à droite sur P->right (Q)
Rotation à gauche sur P

Illustration pas à pas : rotation à droite sur Q

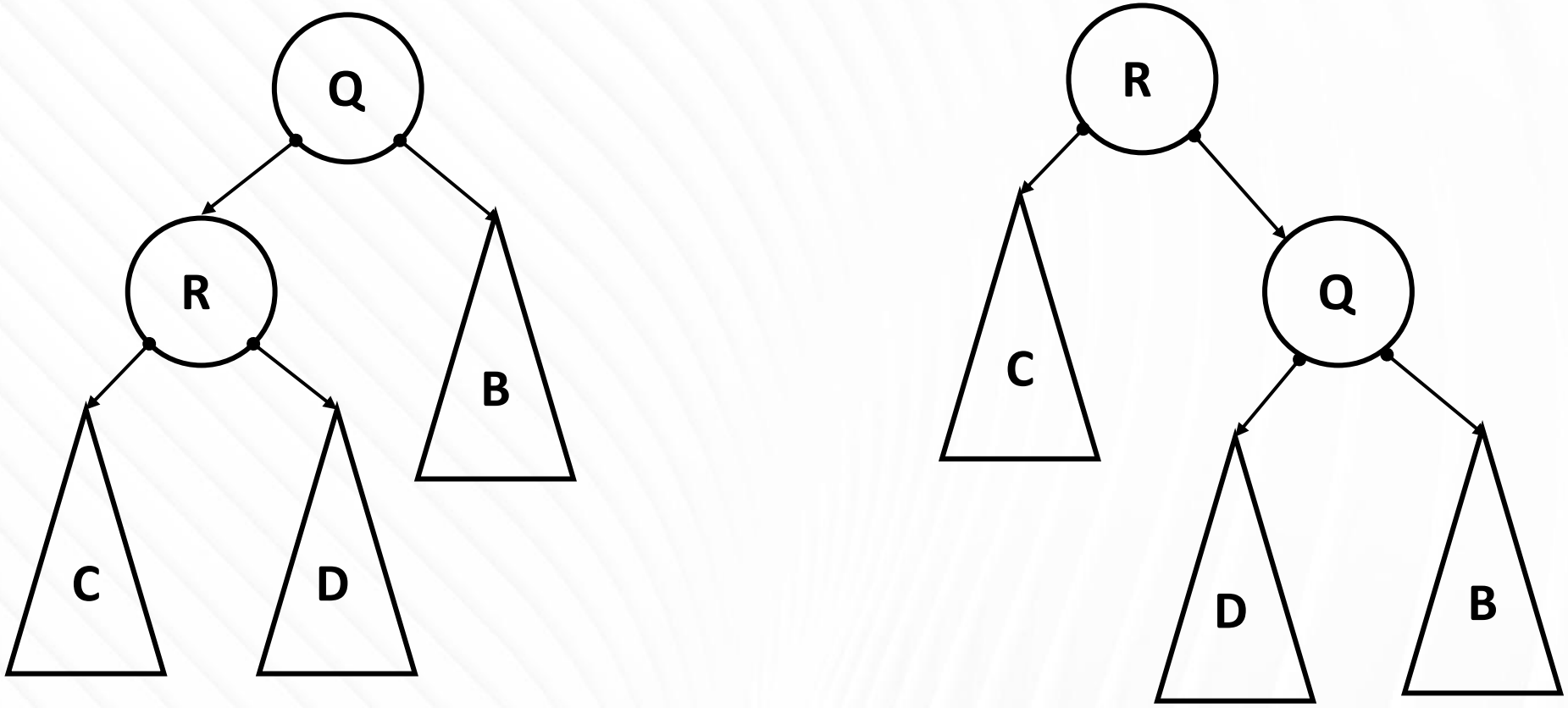
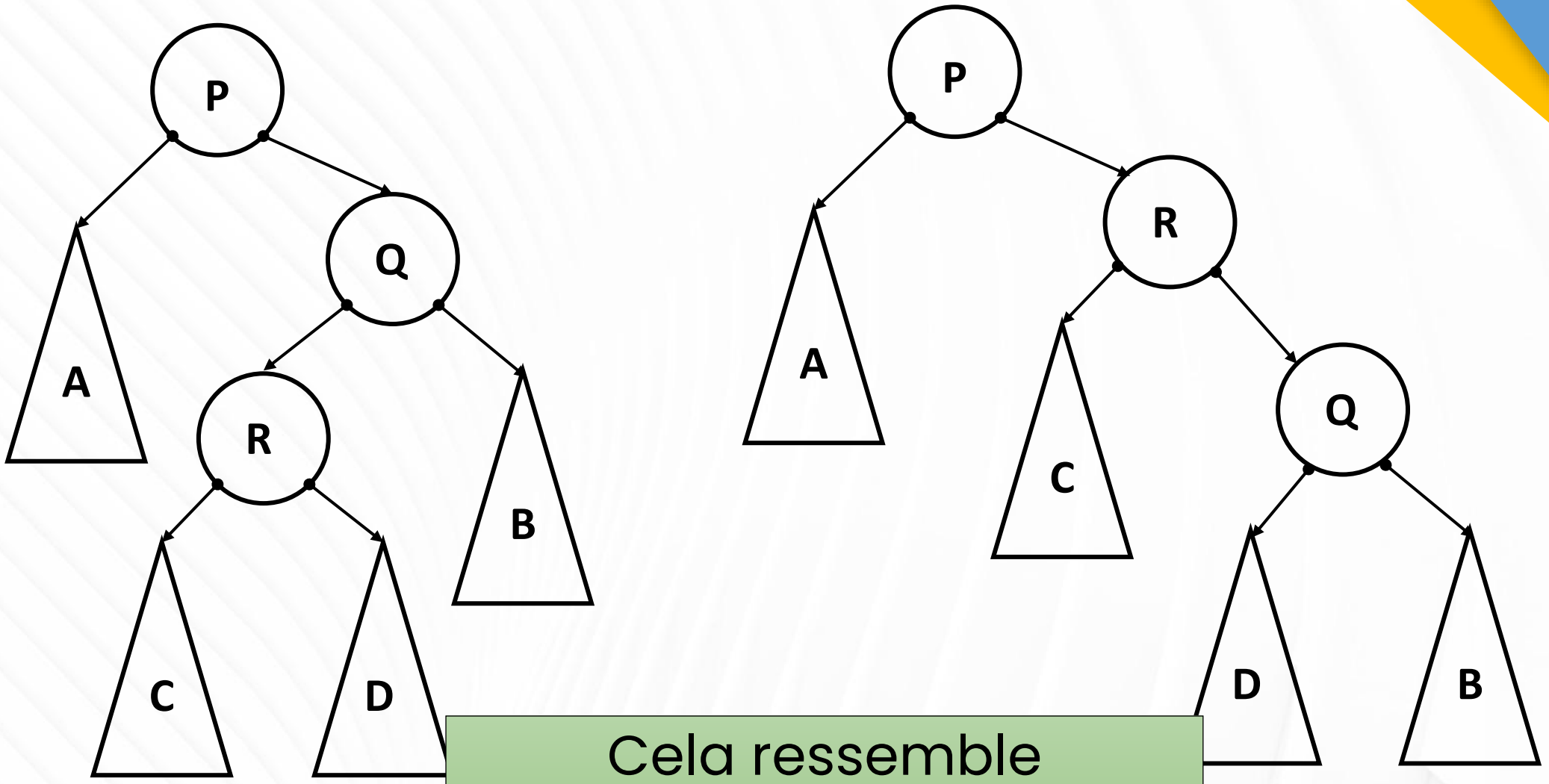
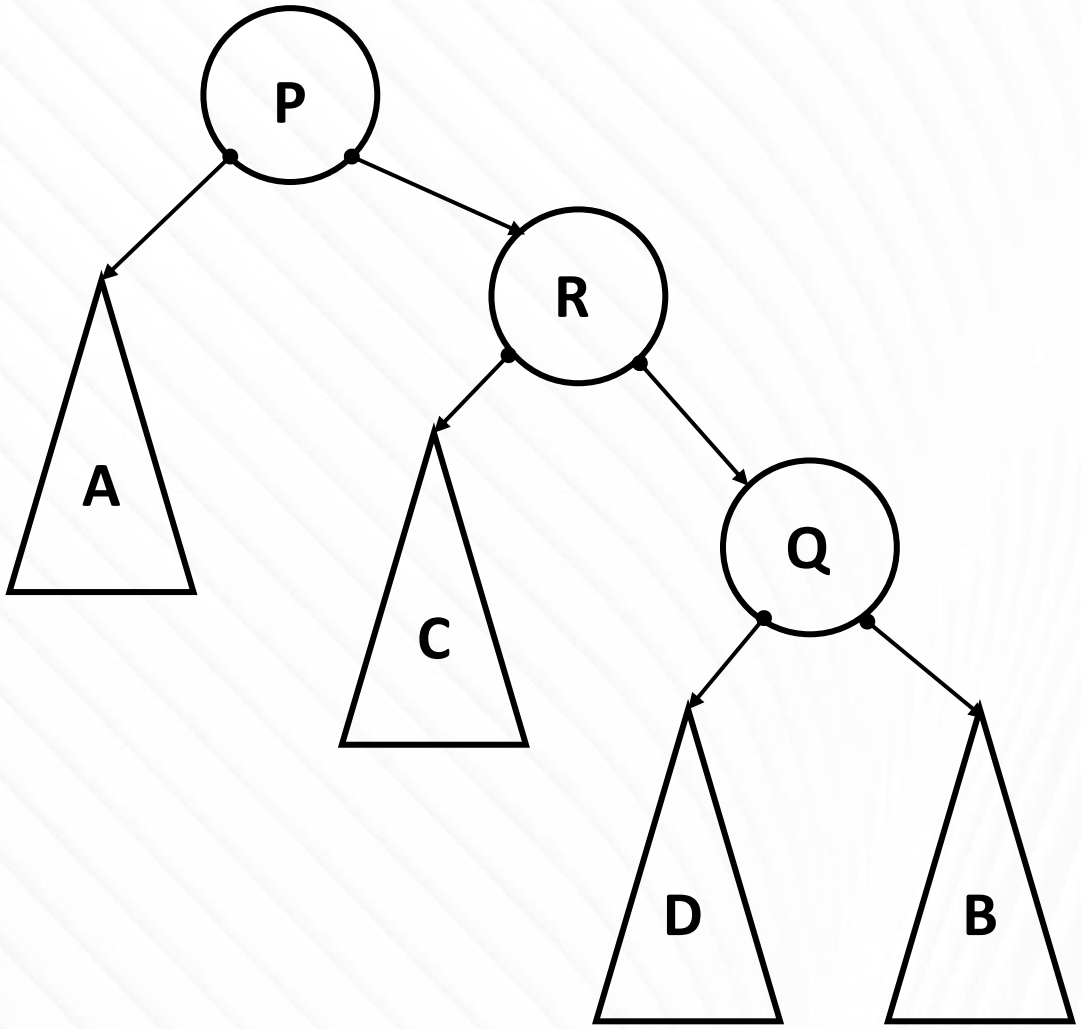


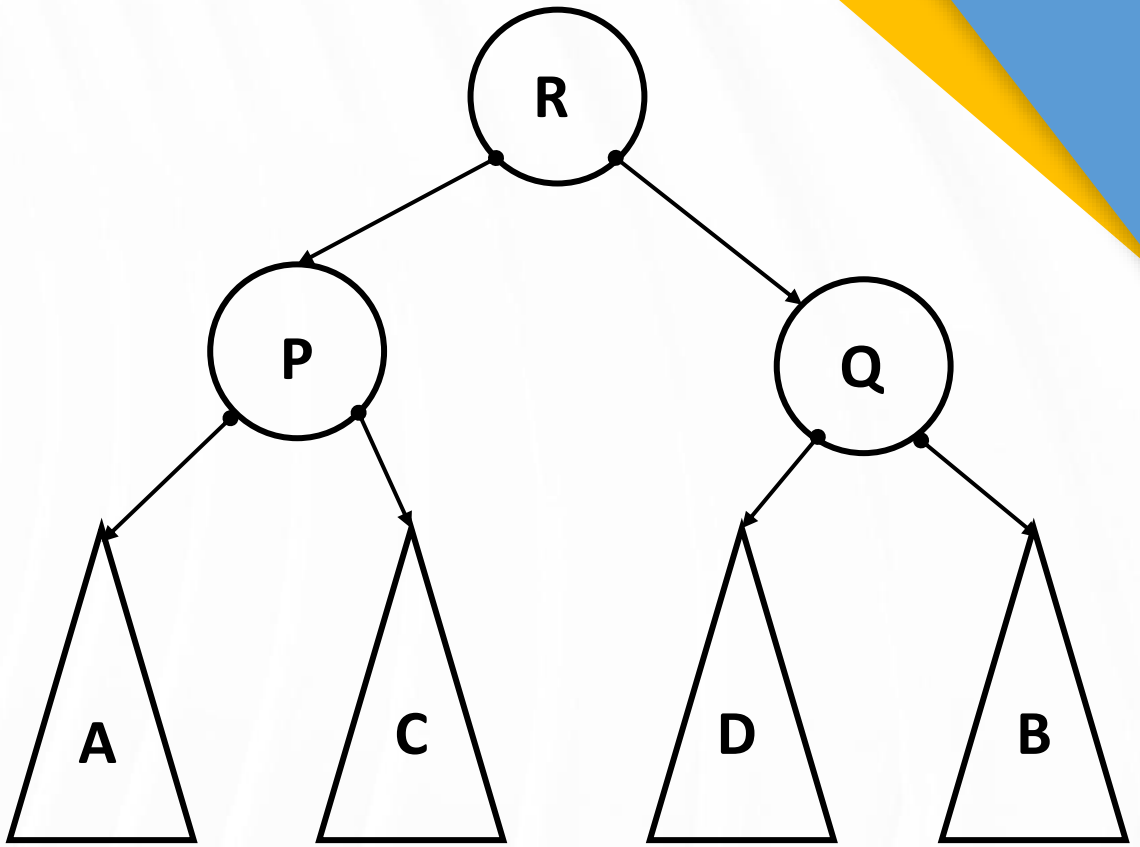
Illustration étape par étape : remplacement dans l'ensemble de l'arbre



Cela ressemble maintenant à l'exemple 1 !

Dernière rotation à gauche sur P





Transformations symétriques, si le facteur d'équilibre est de +2

Facteur d'équilibre d'un nœud **pn**	Facteur d'équilibre du fils	Opération(s)
-2	pn->right : -1	rotation à gauche sur pn
-2	pn->right : +1	rotation à droite sur pn->right, rotation à gauche sur pn
+2	pn->left : -1	rotation à gauche sur pn->left, rotation à droite sur pn
+2	pn->left : +1	rotation à droite sur pn

Mise en œuvre

Insérer un nouveau nœud dans une BST

Calculer les facteurs d'équilibre de manière récursive, jusqu'à trouver (ou non) un nœud dont le facteur d'équilibre est égal à +2 ou -2.

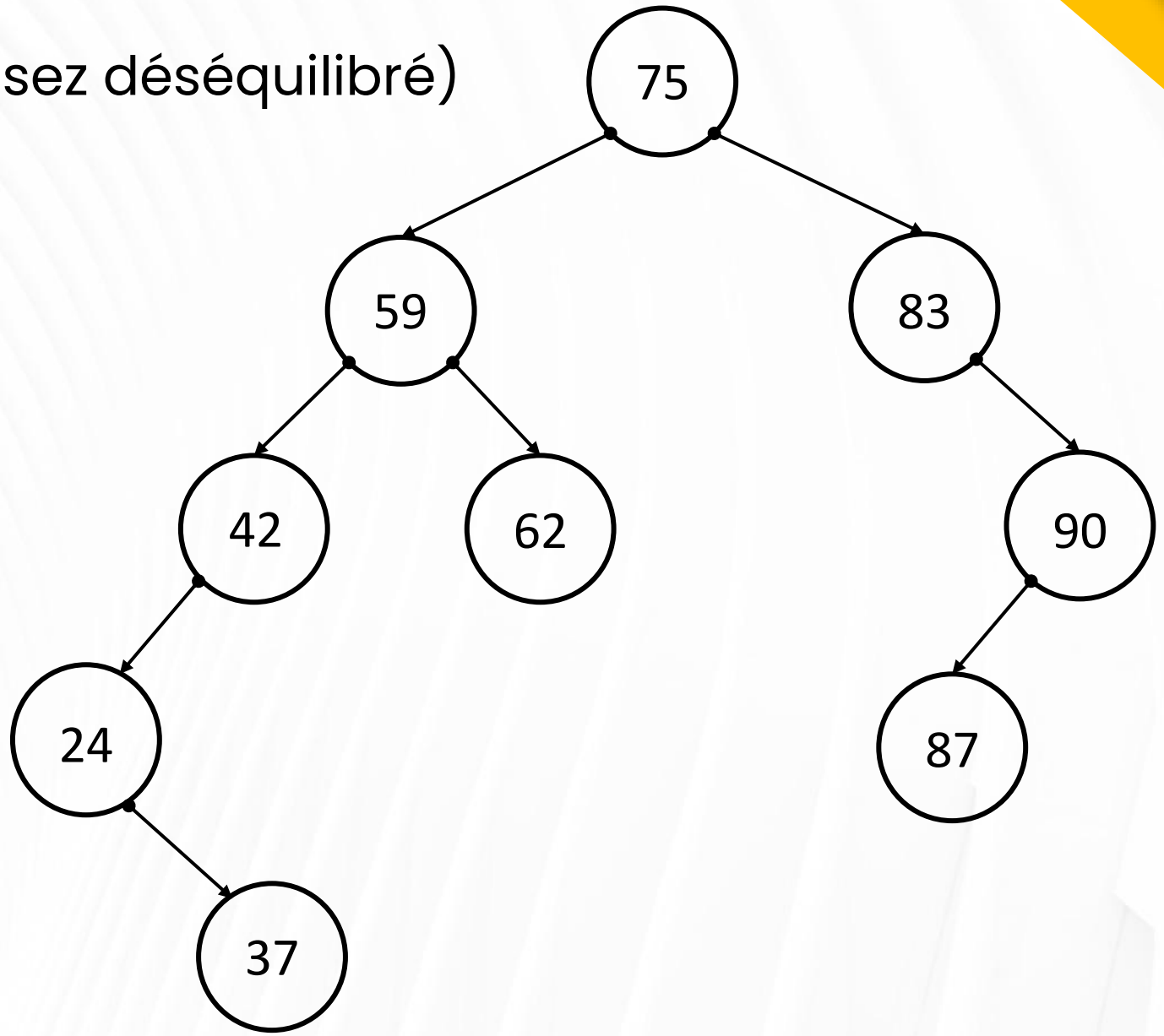
Procéder à l'(aux) opération(s) d'équilibrage, en fonction de la configuration locale (valeurs bf – facteurs d'équilibrage)

Si le BST était un AVL avant cette seule insertion, l'équilibrage est terminé (fin de la récursivité).

Si le BST n'était pas une AVL (exemple : équilibrage d'une BST ordinaire), poursuivre la récursion jusqu'à la racine de l'arbre.

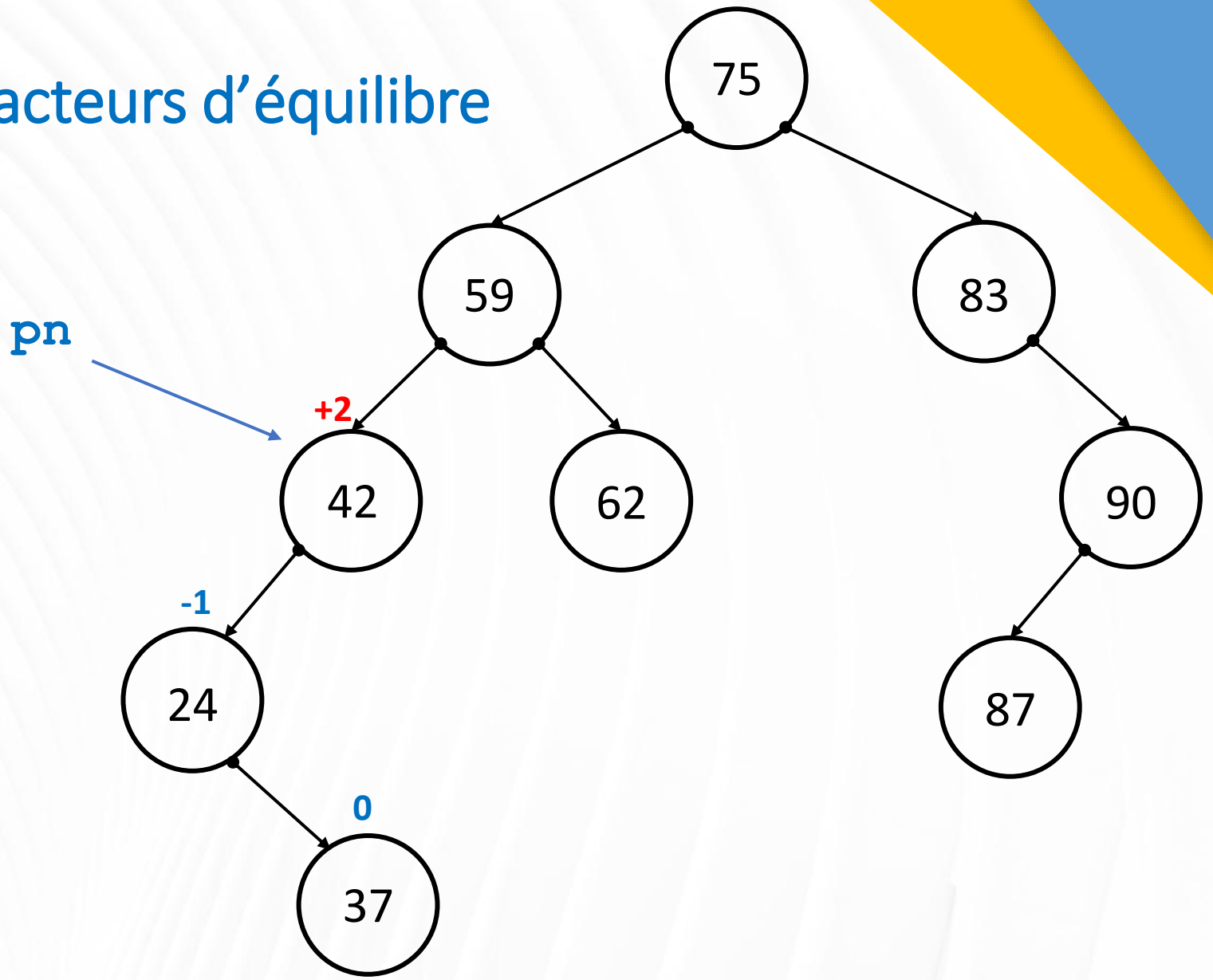
Un exemple (à considérer comme un exercice)

Soit le BST suivant (assez déséquilibré)



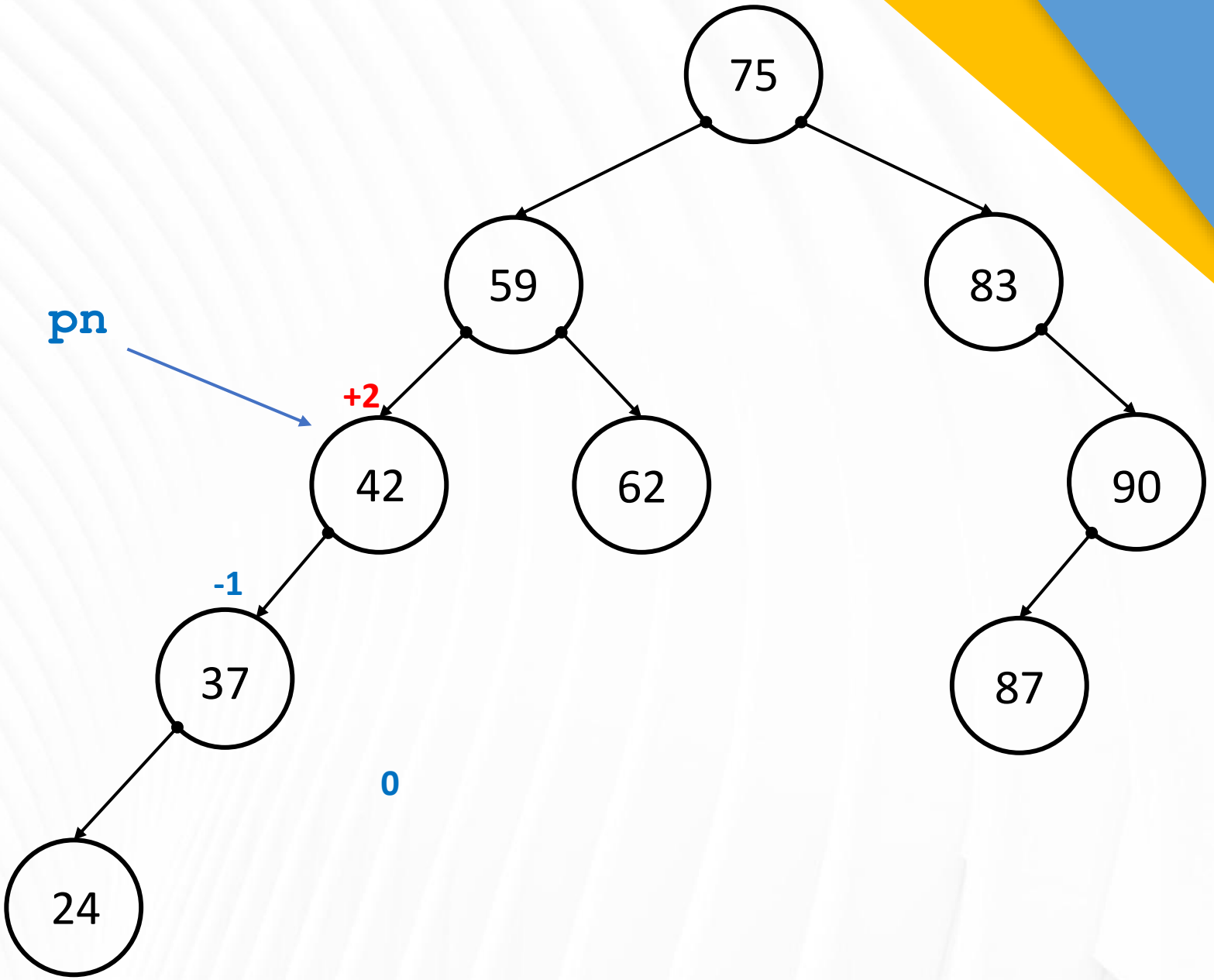
Étape 1 : calcul des facteurs d'équilibre

Stop !
+2 / -1 :
rotation à gauche sur pn->gauche,
rotation à droite sur le pn



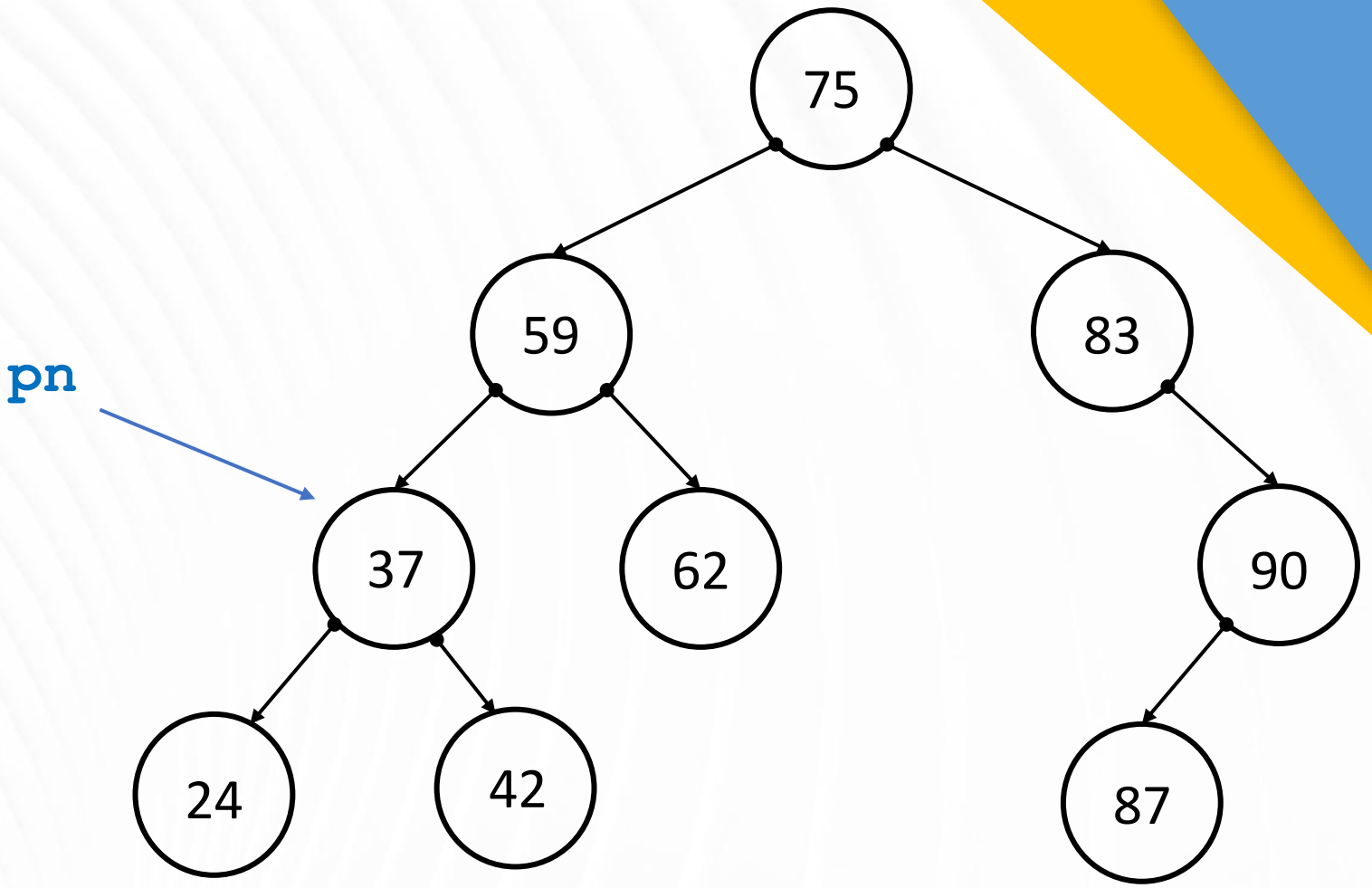
Étape 1 : suite

rotation à gauche sur pn->gauche,



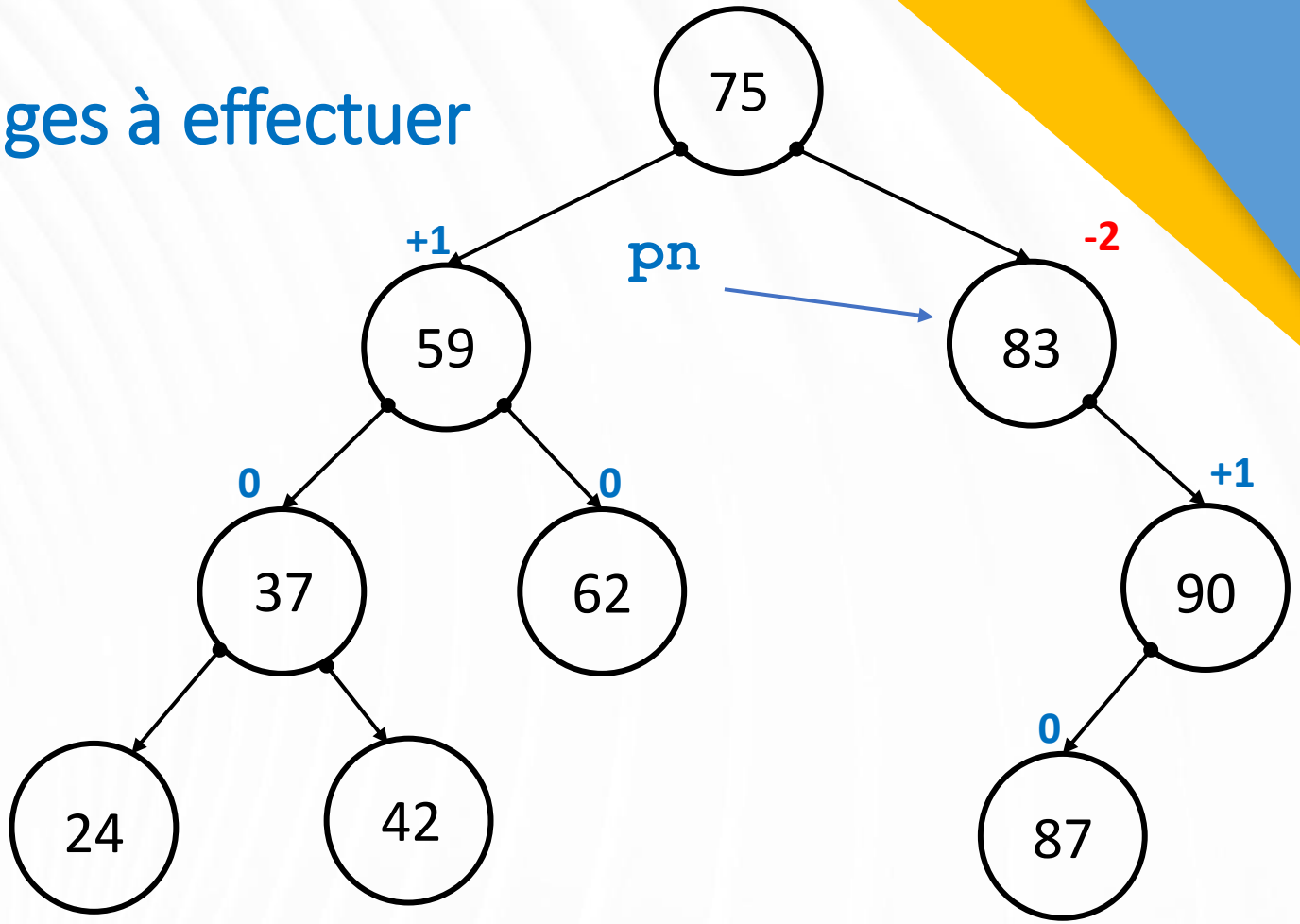
Étape 1 : fin

rotation à droite sur pn



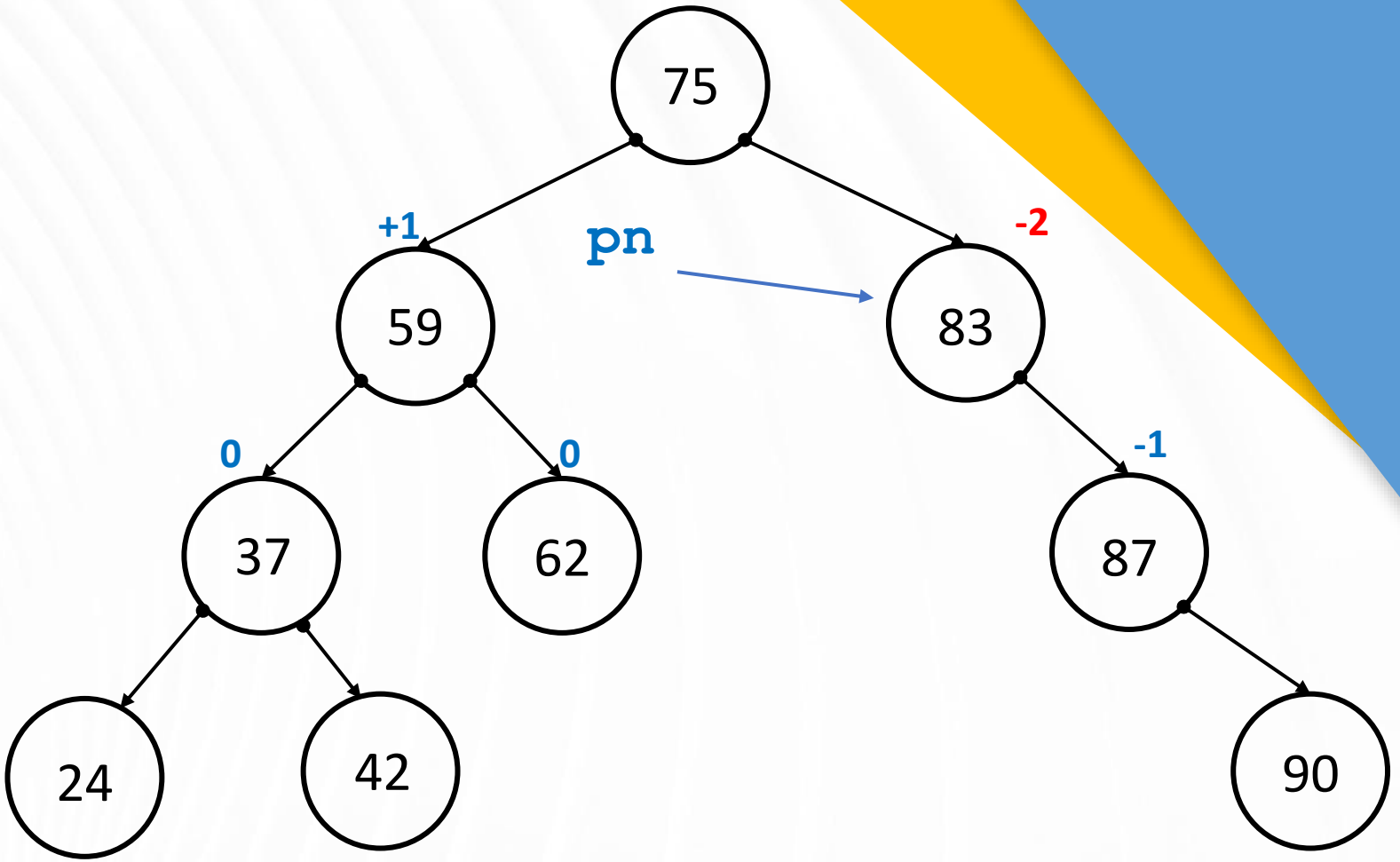
Étape 2 : autres équilibrages à effectuer

Stop !
-2 / +1 :
rotation vers la droite sur pn->right,
rotation à gauche sur le pn

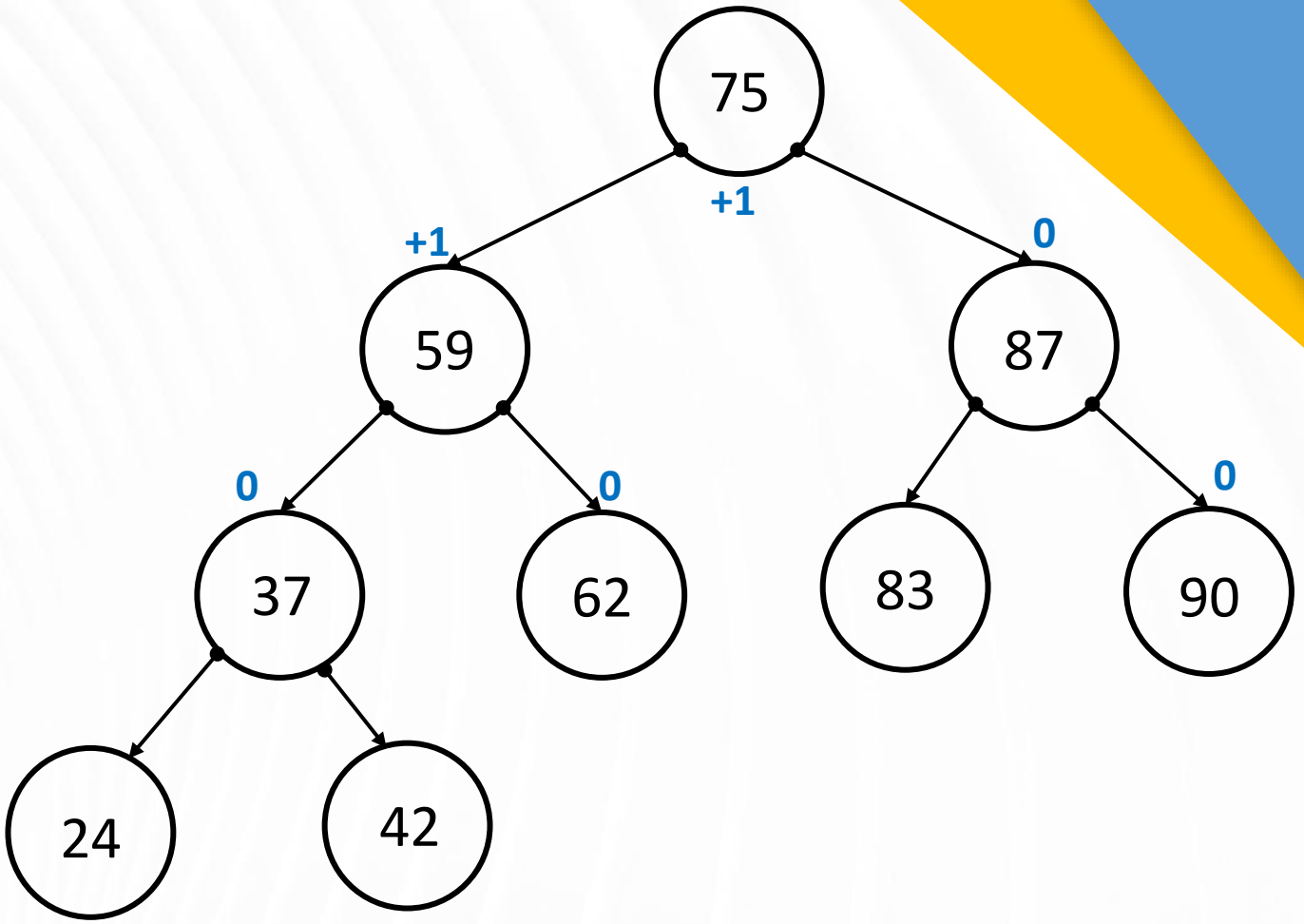


Étape 2 : suite

rotation vers la droite sur pn->right,



Résultat final



Supprimer un nœud dans un AVL

Objectif : garder la structure d'AVL

Description de la méthode :

On cherche le nœud à supprimer

Si c'est une feuille, on le **supprime**

Sinon, s'il n'a qu'un seul fils : on le **remplace** par ce fils

Sinon : on cherche un nœud qui peut le **remplacer**

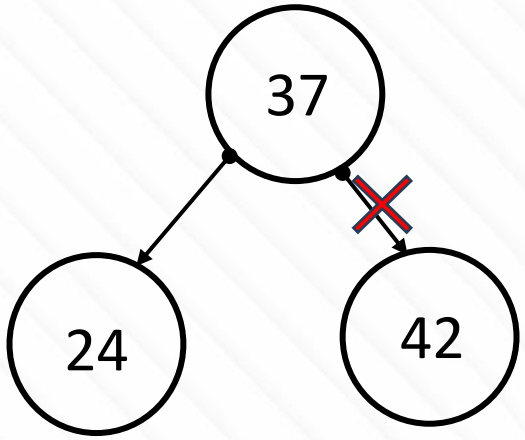
Une fois supprimé, on rééquilibre

Application pour le cas ‘simple’ : le nœud est une feuille

Supprimer un nœud : pas uniquement `free()` sur son adresse

Exemple : supprimer le nœud qui stocke 42

On doit aussi supprimer le lien entre 37 et 42 car 42 est le fils droit de 37



Pour supprimer un nœud de l’arbre, on doit traiter son nœud parent

Cas d'une feuille : tests multiples

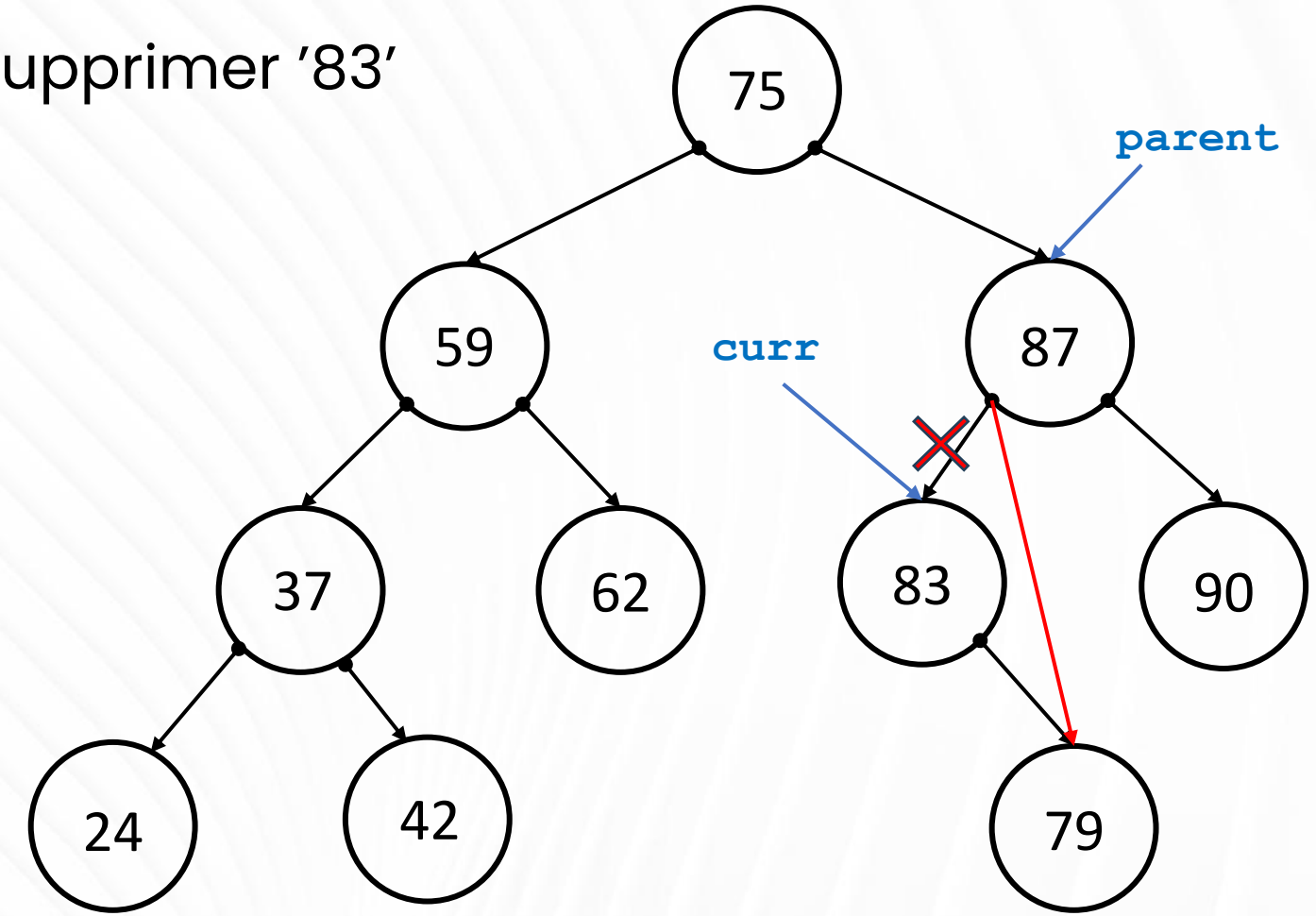
On suppose qu'on a cherché la valeur '42', et que l'on a :

```
t_node *curr;   // pointe sur le nœud
t_node *parent; // pointe sur son parent

Si c'est une feuille :
    s'il s'agit du nœud root, alors mettre root à NULL
    sinon :
        si curr est le fils gauche de son parent
                mettre le fils gauche du parent à NULL
        sinon
                mettre le fils droit du parent à NULL
    libérer le nœud pointé sur curr
```

Cas d'un seul fils

Illustration : supprimer '83'



Cas d'un seul fils

```
t_node *curr;  // pointe sur le nœud
t_node *parent; // pointe sur son parent
```

On doit déterminer :

Si le nœud fils de curr est son fils gauche ou droit

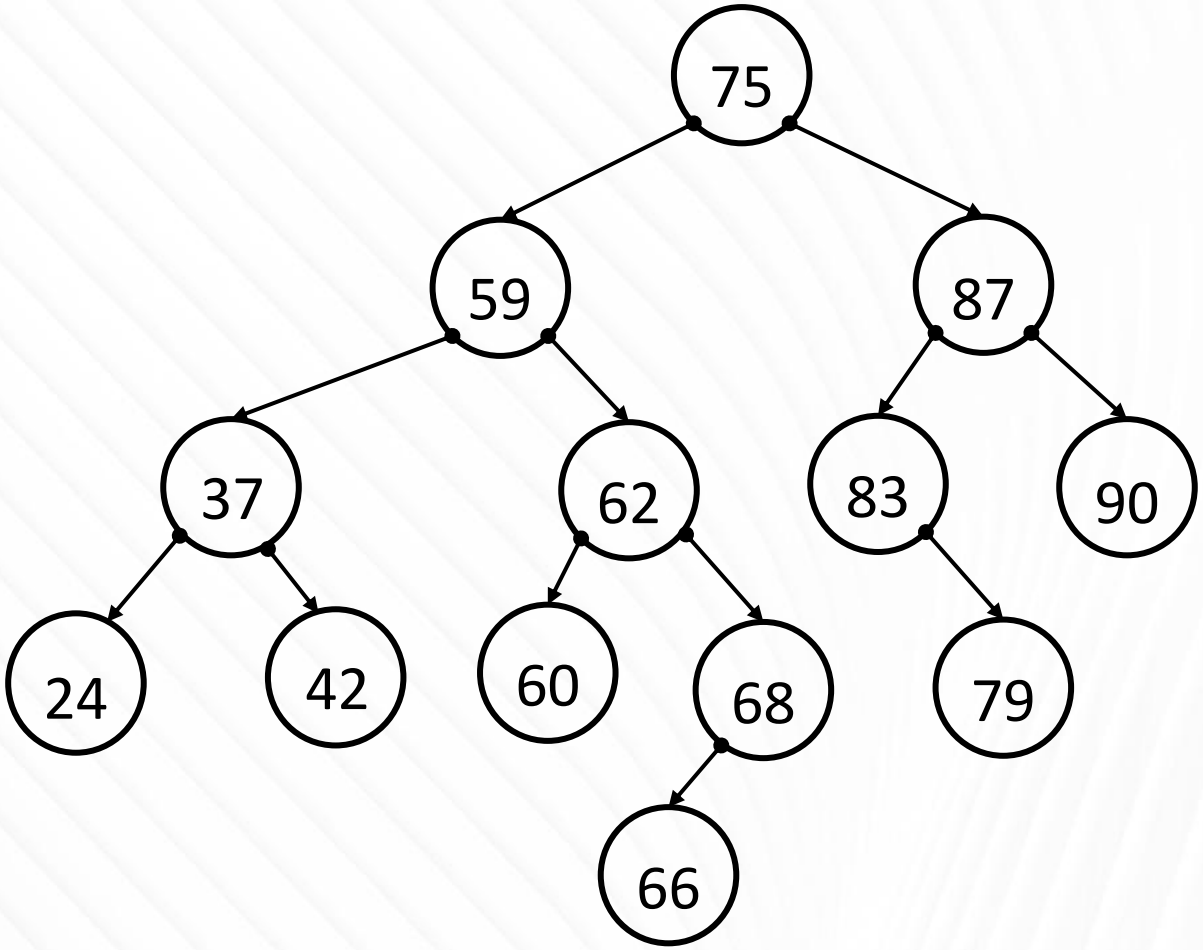
Si curr est le fils gauche ou droit de son parent

Puis attacher ce nœud fils en tant que fils gauche ou droit du parent

Et enfin libérer le nœud curr

Cas de deux fils

Illustration : supprimer ‘75’ (ou ’87’, ou ’37)



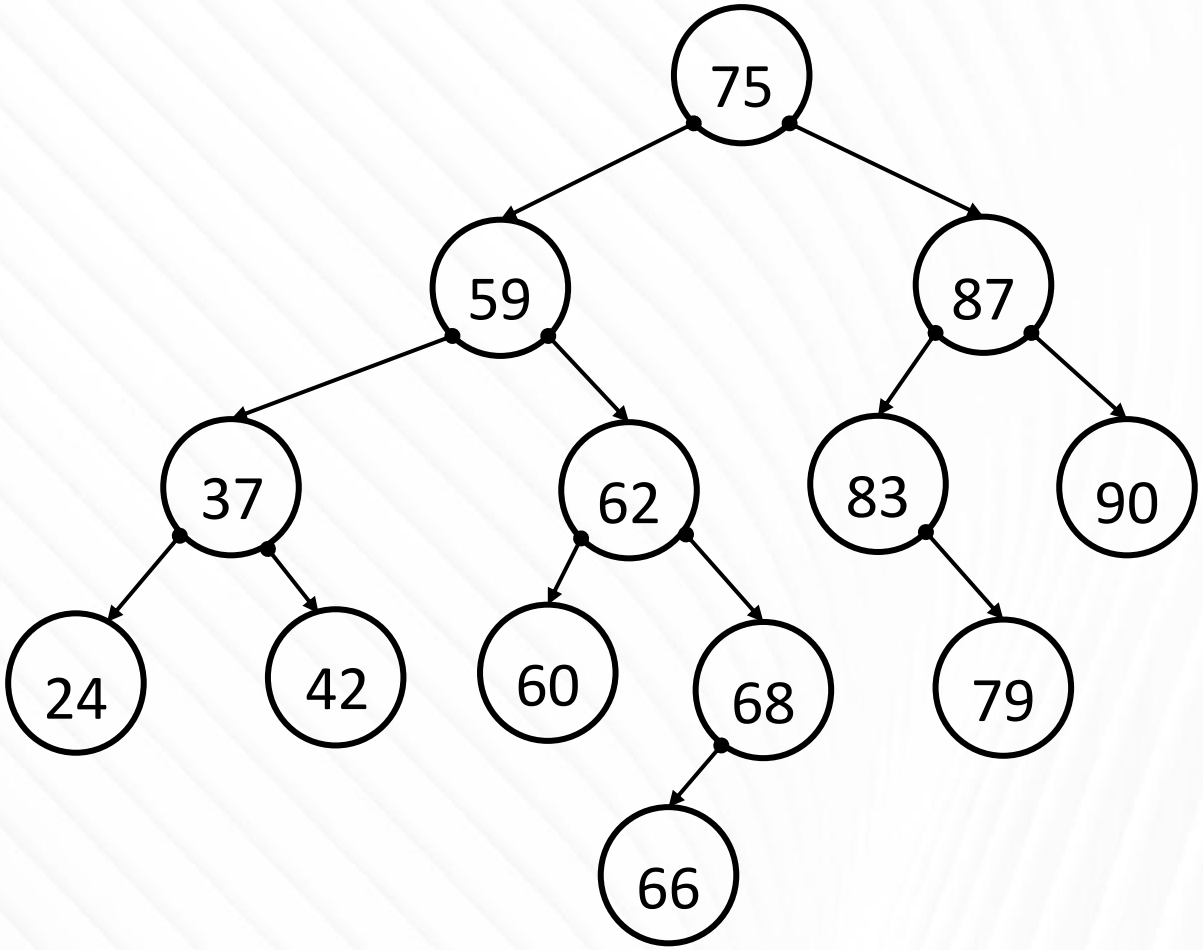
Pas de solution immédiate pour ‘recoller’ les morceaux si on supprime directement le nœud

Idée : trouver un ‘remplaçant’ qu’on pourra supprimer

Contraintes sur ce remplaçant :

- Doit avoir 0 ou 1 fils (on sait faire)
- Doit conserver la règle BST

Cas de deux fils : recherche du remplaçant



Règle BST : plus petit à gauche, plus grand à droite

Constat : **la valeur maximale dans le sous-arbre gauche du nœud à supprimer peut le remplacer**.

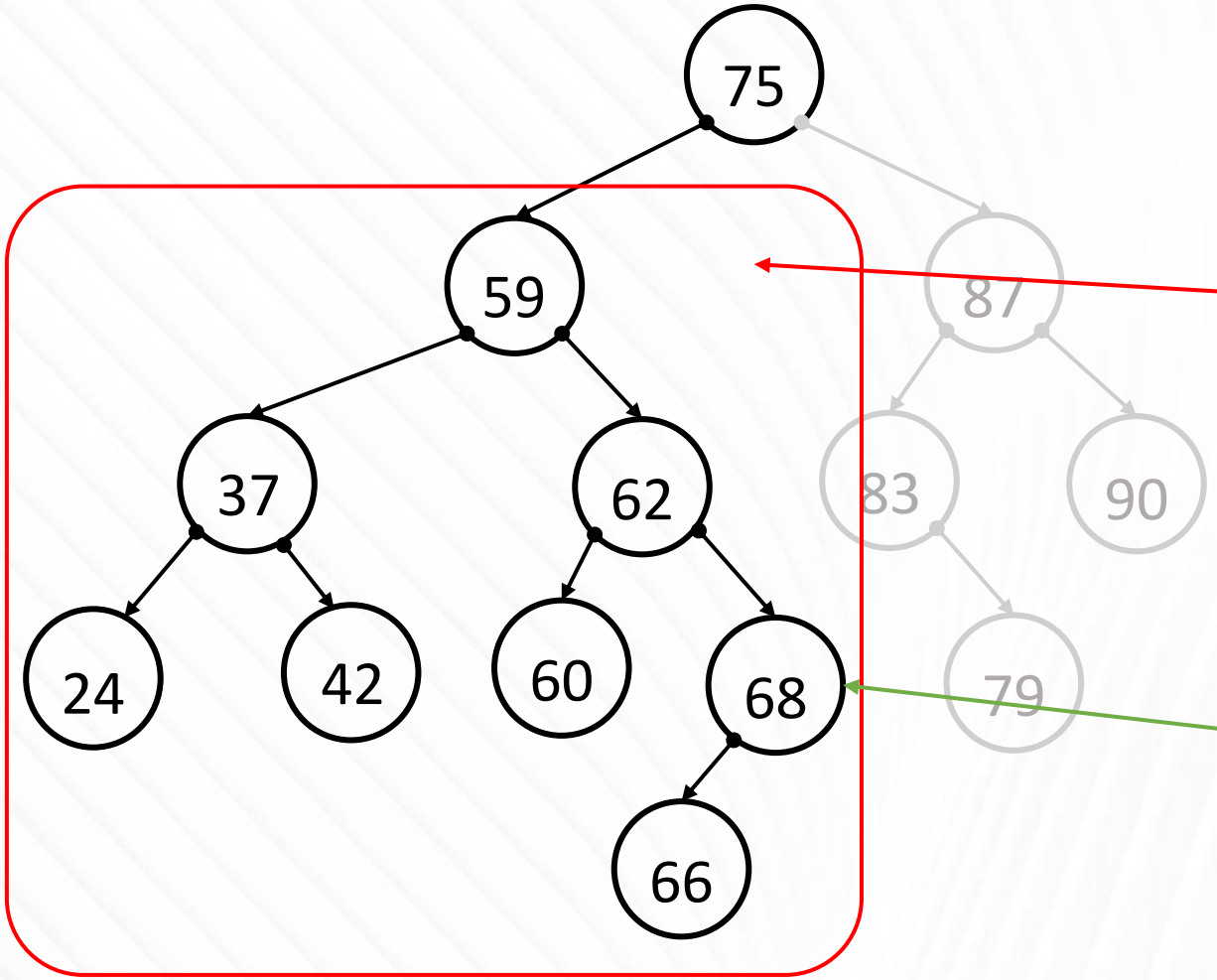
Pourquoi ?

Elle est plus grande que toutes les autres (c'est le maximum)

Le nœud qui la stocke n'a pas de fils droit (sinon il stockerait une valeur plus grande)

Cas de deux fils : recherche du remplaçant - méthode

Illustration : supprimer ‘75’

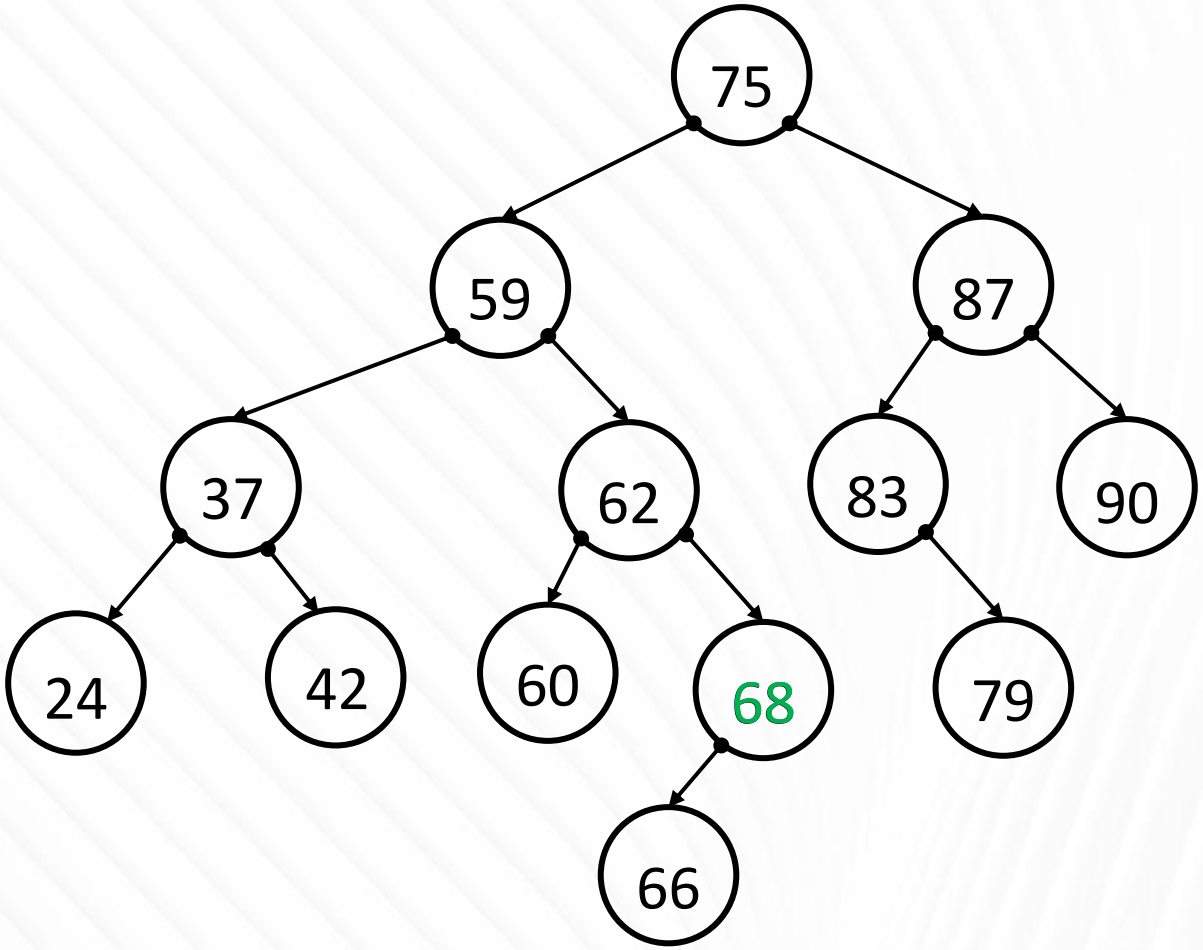


Ce maximum est ‘le plus à droite possible’ dans le sous arbre gauche du nœud à supprimer

On part de la racine de ce sous-arbre et on va tout le temps à droite

On trouve notre remplaçant, c’est 68

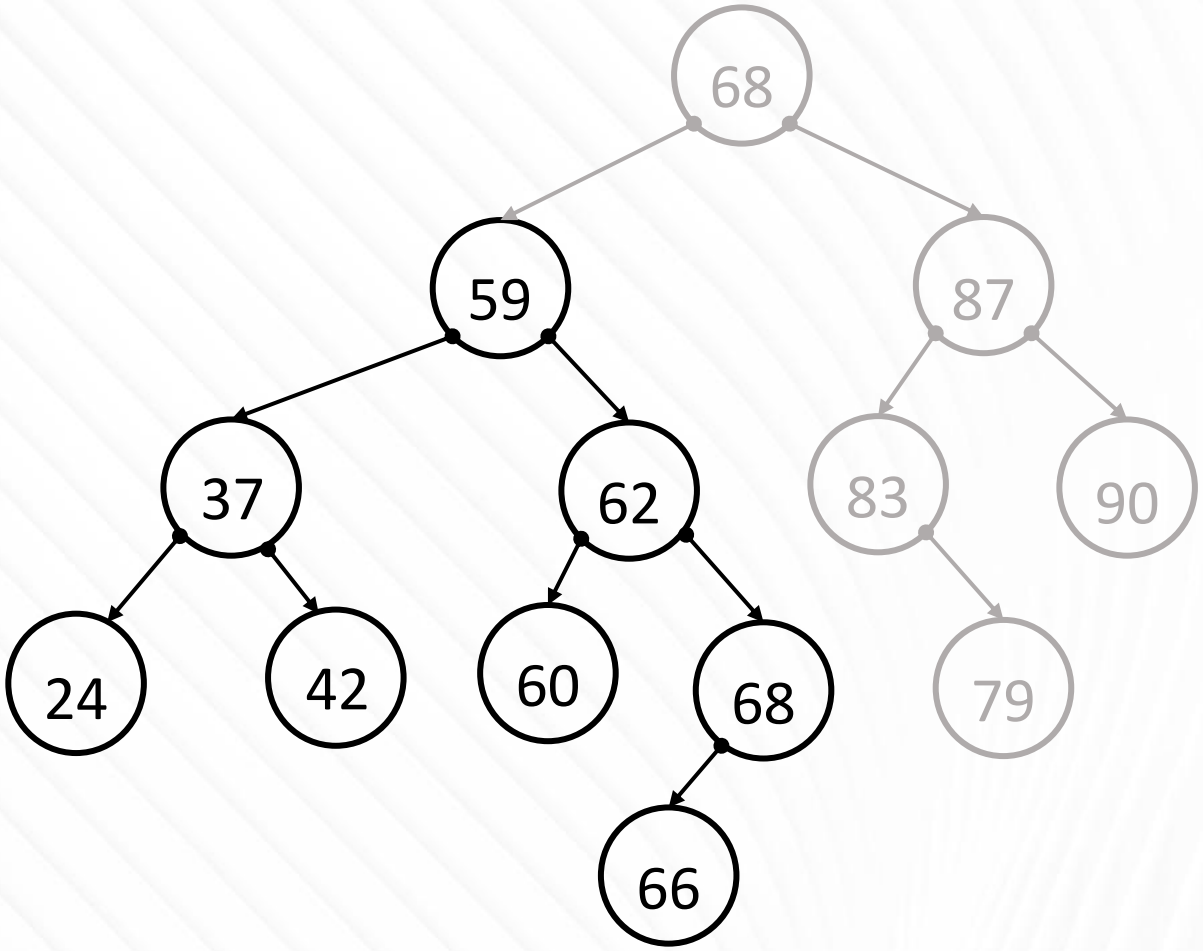
Remplacement / suppression



On copie la valeur du remplaçant dans le nœud à ‘supprimer’

On relance une suppression de cette valeur copiée dans le sous-arbre gauche du nœud à ‘supprimer’

Seconde suppression



Un résumé complet avec les rééquilibrages

Une vidéo très intéressante :

plusieurs cas de suppression

Une vidéo sur une structure plus sophistiquée : B-trees

Une vidéo très intéressante :

B-trees